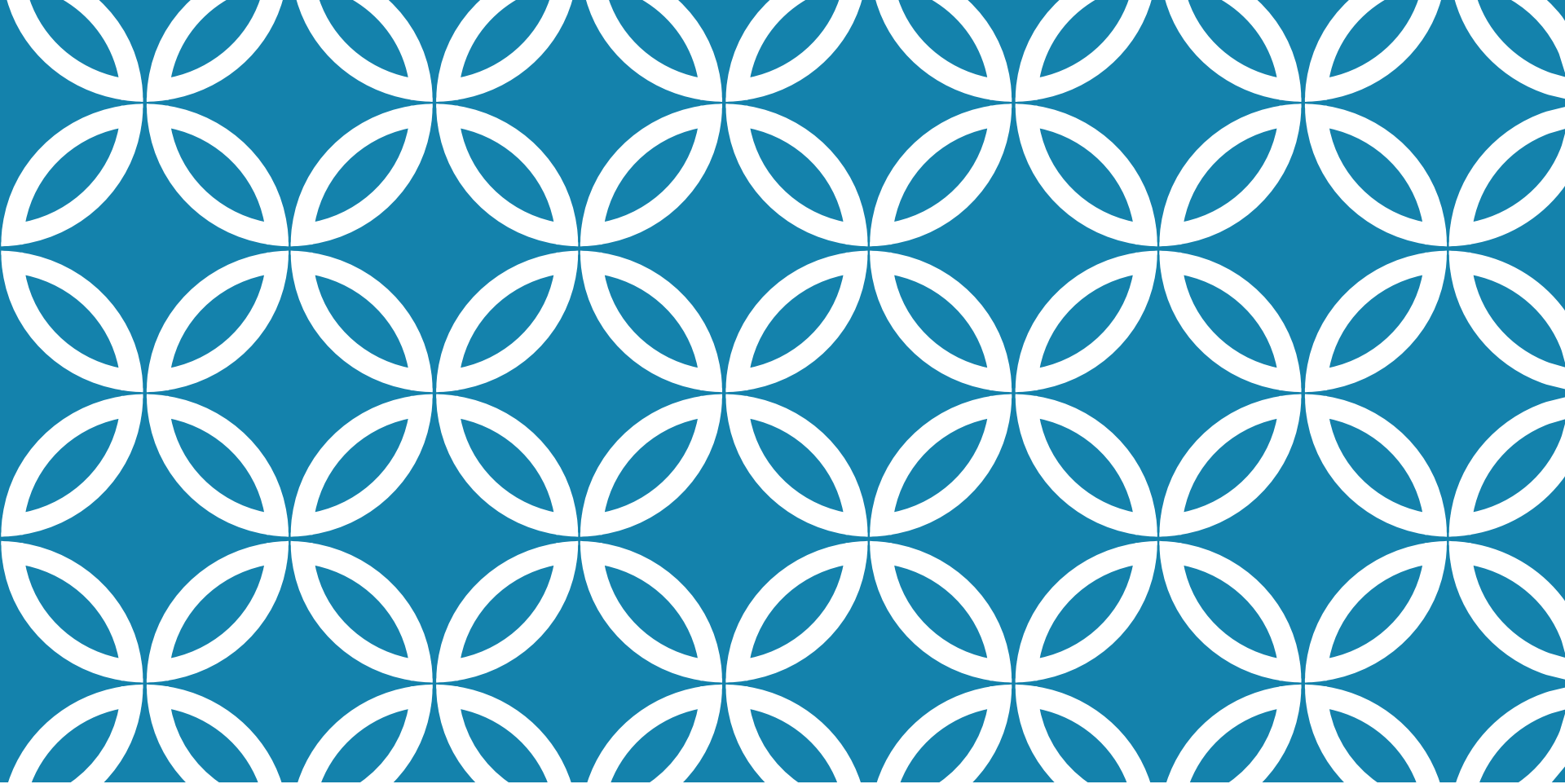




CHAPTER 3

**Image Transform:
Frequency Domain
Representation and
Enhancement**

SYLLABUS

Image Transform: Frequency Domain Representation and Enhancement (10-CO3)

3.1 Introduction , DFT and its properties, radix-2 algorithm(2- DFT), FFT algorithm: divide and conquer approach, Dissemination in Time(DIT)-FFT

3.2 Discrete Cosine Transform, Walsh Transform, Hadamard Transform, Haar Transform, Principal component Analysis(PCA/Hoteling Transform), Introduction to Wavelet Transform

3.3 Low Pass and High Pass Frequency domain filters: Ideal, Butterworth, Homomorphic filter

Self-Learning Topic: Discrete Sine Transform (DST)

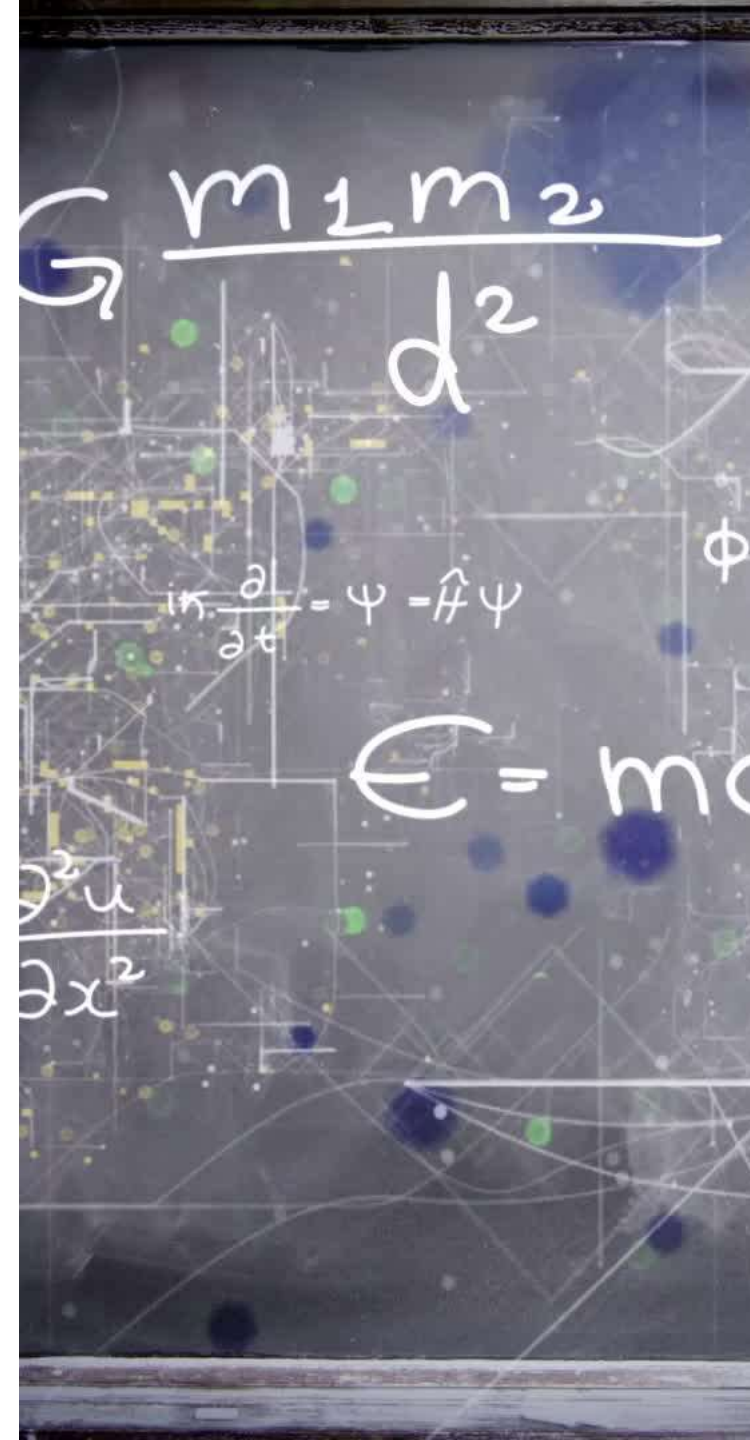


IMAGE TRANSFORMS

- Extensively used in image processing and image analysis.
- Transform is basically a mathematical tool, which allows us to move from one domain to another domain (time domain to the frequency domain).
- Transforms change the representation of a signal by projecting it onto a set of basis functions.
- The transforms do not change the information content present in the signal.

NEED FOR TRANSFORM

(i) Mathematical Convenience:

Every action in time domain will have an impact in the frequency domain.

The complex convolution operation in time domain is equal to simple multiplication operation in the frequency domain

(ii) To Extract more Information:

Transforms allow us to extract more relevant information.

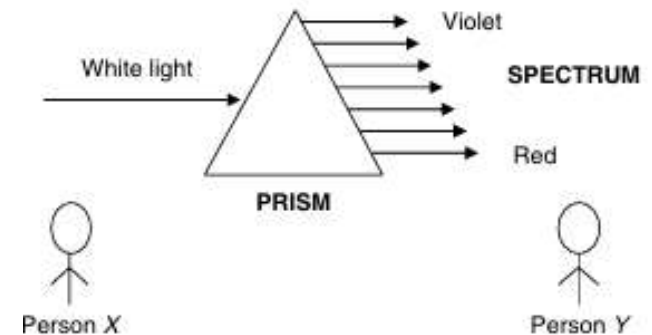


Fig. 4.1 Spectrum of white light



Fig. 4.2 Concept of transformation

CLASSIFICATION OF IMAGE TRANSFORMS

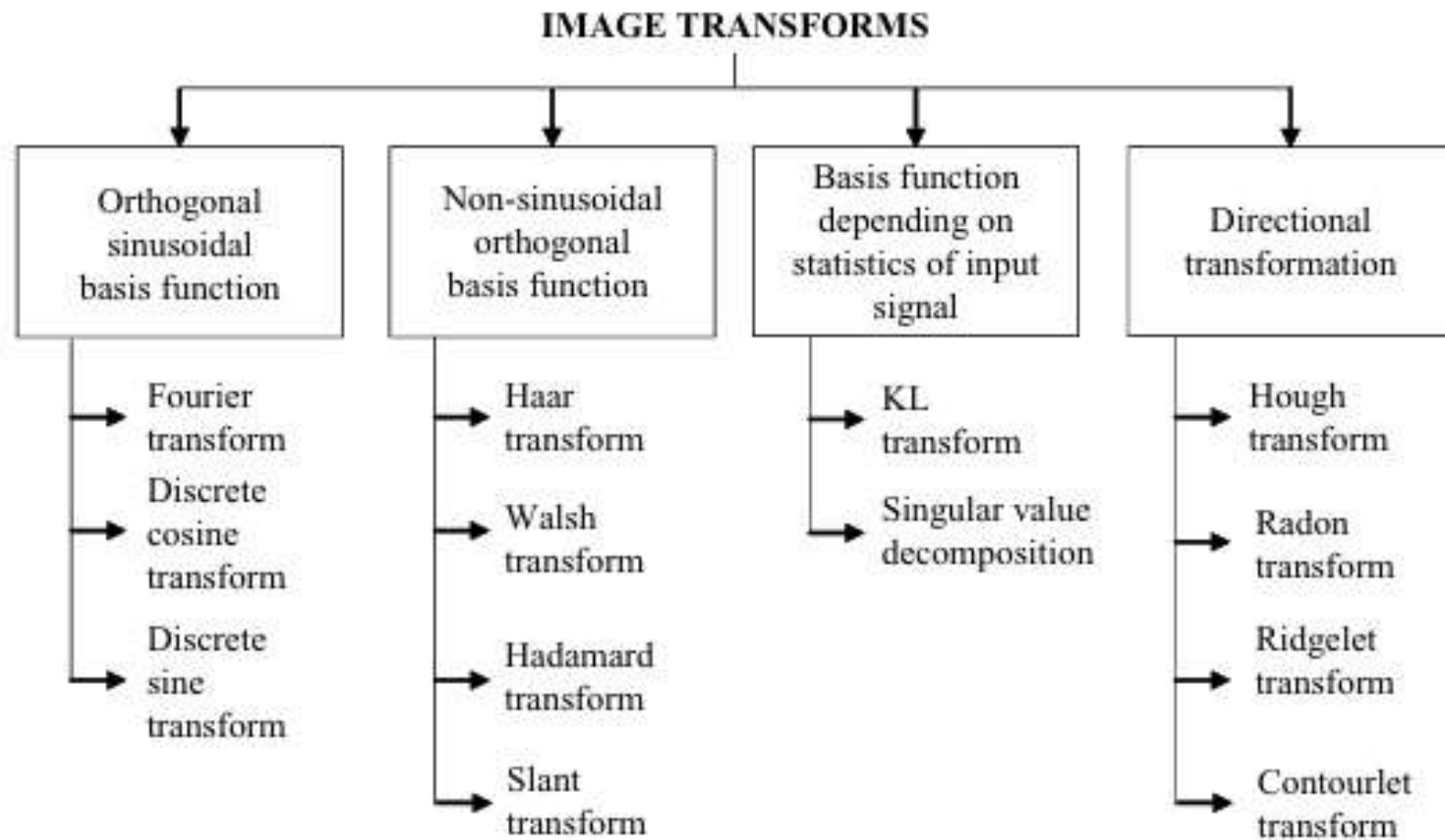


IMAGE TRANSFORM

1) Orthogonal sinusoidal basis function

One of the most powerful transforms with orthogonal sinusoidal basis function is the Fourier transform.

The transform which is widely used in the field of **image compression** is discrete cosine transform.

IMAGE TRANSFORM

2) Non-sinusoidal orthogonal basis function

The transforms whose basis functions are non-sinusoidal in nature.

One of the important advantages of wavelet transform is that signals (images) can be **represented in different resolutions**.

IMAGE TRANSFORM

3) Basis function depending on statistics of input signal

The transform whose basis function depends on the statistics of input signal are KL transform and singular value decomposition.

The KL transform is considered to be the best among all linear transforms with respect to **energy compaction**.

IMAGE TRANSFORM

4) Directional transformation

The transforms whose basis functions are effective in **representing the directional information of a signal** include Hough transform, Radon transform, Ridgelet transform and Contourlet transform.

FOURIER TRANSFORM

Fourier transform is widely used in the field of image processing.

A Fourier transform is used to transform an intensity image into the domain of spatial frequency.

For a continuous time signal $x(t)$, the Fourier transform is defined as $X(\Omega)$

$$x(t) \xrightarrow{\text{CTFT}} X(\Omega)$$

Continuous Time Fourier Transform (CTFT) is defined as

$$X(\Omega) = \int_{-\infty}^{\infty} x(t) e^{-j\Omega t} dt$$

A continuous time signal $x(t)$ is converted into discrete time signal $x(nT)$ by sampling process, where T is the sampling interval

The Fourier transform of a finite energy discrete time signal $x(nT)$ is given by

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(nT) e^{-j\Omega nT}$$

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(n) e^{-j(\Omega T)n}$$

A continuous time signal $x(t)$ is converted into discrete time signal $x(nT)$ by sampling process, where T is the sampling interval

The Fourier transform of a finite energy discrete time signal $x(nT)$ is given by

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(nT) e^{-j\Omega nt}$$

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x(n) e^{-j(\Omega T)n}$$

Where $X(e^{j\omega})$ is known as Discrete-Time Fourier Transform (DTFT) and is a continuous function of ω . The relation between ω and Ω is given by

$$\omega = \Omega T$$

$$\omega = 2\pi f \times T$$

where T is the sampling interval and is equal to $1/f_s$. Replacing T by $1/f_s$ in above equation we get

$$\omega = 2\pi f \times \frac{1}{f_s}$$

$$\frac{f}{f_s} = k$$

$$\omega = k \times 2\pi$$

To limit the infinite number of values to a finite number, above equation is modified as $\omega/2\pi = k/N$

$$X(K) = \sum_{n=0}^{N-1} x(n) e^{-j\frac{2\pi}{N}kn}$$

where $k = 0, 1, \dots, N-1$

DISCRETE FOURIER TRANSFORM

Discrete Fourier transform for a finite sequence is given by:

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi \frac{kn}{N}}$$

Evaluating summation for finite sequence for 0 to N-1.

k/N corresponds to frequency F

N corresponds to time for discrete signals

INVERSE DISCRETE FOURIER TRANSFORM

The inverse discrete Fourier transform of $X(k)$ is defined as

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N} \quad 0 \leq n \leq N-1$$

Where K and n are in the range of $0, 1, 2, \dots, N-1$ For example, if $N=4$, $K=0, 1, 2, 3$; $N=0, 1, 2, 3$

UNITARY TRANSFORM

Unitary Transform = A discrete linear transform is unitary if its transform matrix conforms to the unitary condition

$$A \times A^H = I$$

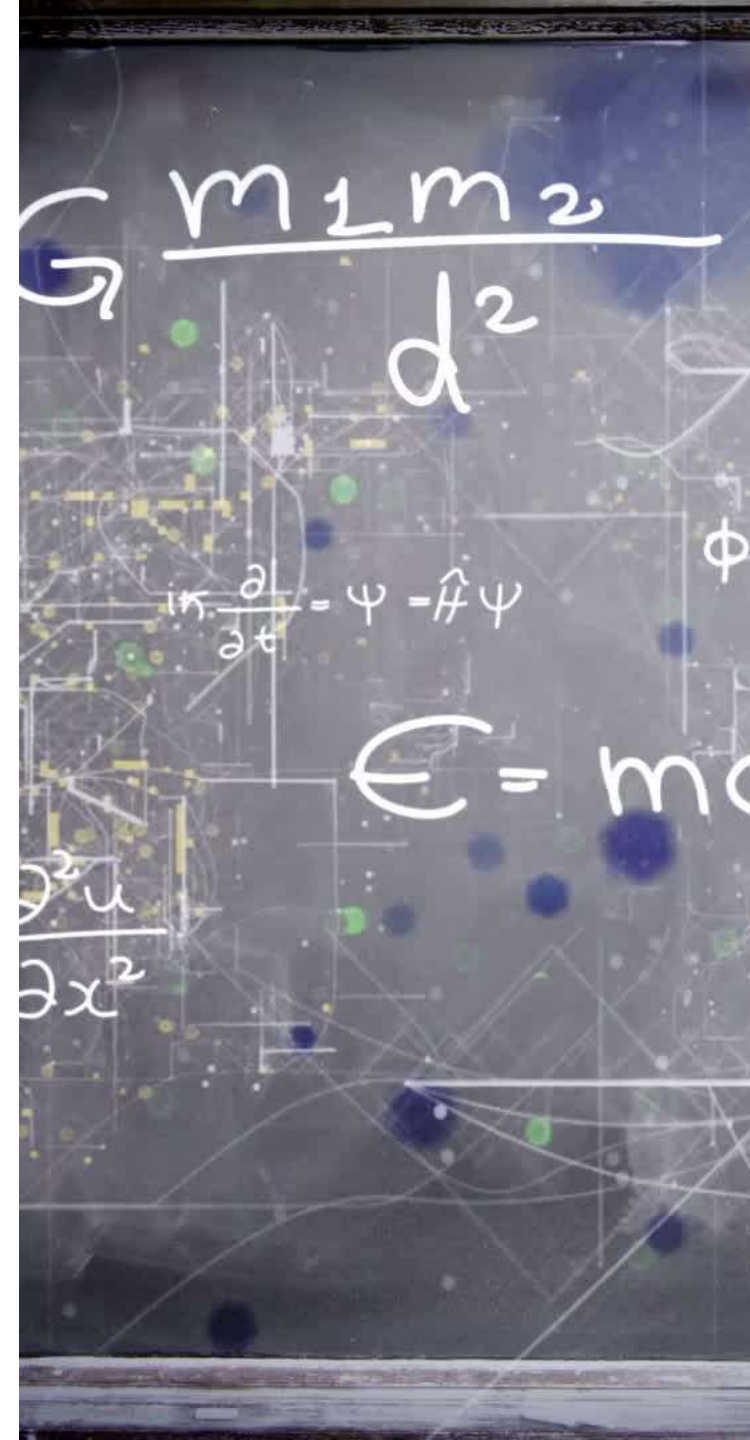
where A = transformation matrix,

A^H represents Hermitian matrix.

$$A^H = A^{*T}$$

I = identity matrix

When the transform matrix A is unitary, the defined transform is called unitary transform



ORTHOGONAL DFT

If A is unitary and has only real elements , then it is orthogonal matrix

A is orthogonal if $A.A' = I$

IMAGE TRANSFORM

1) Check whether the DFT matrix is unitary or not.

Solution

Step 1 Determination of the matrix A

Finding 4-point DFT (where $N = 4$)

The formula to compute a DFT matrix of order 4 is given below.

$$X(K) = \sum_{n=0}^3 x(n) e^{-j \frac{2\pi}{4} kn} \text{ where } k = 0, 1, \dots, 3$$

1. Finding $X(0)$

$$X(0) = \sum_{n=0}^3 x(n) = x(0) + x(1) + x(2) + x(3)$$

2. Finding $X(1)$

$$\begin{aligned}X(1) &= \sum_{n=0}^3 x(n) e^{-j\frac{\pi}{2}n} \\&= x(0) + x(1)e^{-j\frac{\pi}{2}} + x(2)e^{-j\pi} + x(3)e^{-j\frac{3\pi}{2}} \\X(1) &= x(0) - jx(1) - x(2) + jx(3)\end{aligned}$$

3. Finding $X(2)$

$$\begin{aligned}X(2) &= \sum_{n=0}^3 x(n) e^{-j\pi n} \\&= x(0) + x(1)e^{-j\pi} + x(2)e^{-j2\pi} + x(3)e^{-j3\pi} \\X(2) &= x(0) - x(1) + x(2) - x(3)\end{aligned}$$

4. Finding $X(3)$

$$\begin{aligned}X(3) &= \sum_{n=0}^3 x(n)e^{-j\frac{3\pi}{2}n} \\&= x(0) + x(1)e^{-j\frac{3\pi}{2}} + x(2)e^{-j3\pi} + x(3)e^{-j\frac{9\pi}{2}} \\X(3) &= x(0) + jx(1) - x(2) - jx(3)\end{aligned}$$

Collecting the coefficients of $X(0)$, $X(1)$, $X(2)$ and $X(3)$, we get

$$X[k] = A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

Step 2 Computation of A^H

To determine A^H , first determine the conjugate and then take its transpose.

$$A \xrightarrow{\text{Conjugate}} A^* \xrightarrow{\text{Transpose}} A^H$$

Step 2a Computation of conjugate A^*

$$A^* = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & j & -1 & -j \\ 1 & -1 & 1 & -1 \\ 1 & -j & -1 & j \end{bmatrix}$$

Step 2b Determination of transpose of A^*

$$(A^*)^T = A^H = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & j & -1 & -j \\ 1 & -1 & 1 & -1 \\ 1 & -j & -1 & j \end{bmatrix}$$

Step 3 Determination of $A \times A^H$

$$A \times A^H = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & j & -1 & -j \\ 1 & -1 & 1 & -1 \\ 1 & -j & -1 & j \end{bmatrix} = \begin{bmatrix} 4 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix} = 4 \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The result is the identity matrix, which shows that Fourier transform satisfies unitary condition.

UNITARY TRANSFORM

unitary transformation preserves the **signal energy**

Most unitary transforms pack a **large fraction of the energy of the image into relatively few of the transform coefficients.**

Few of the transform coefficients have significant values and these are the coefficients that are close to the origin

PROPERTIES OF DFT-

Periodicity

$$x(n) \xrightleftharpoons[N]{\text{DFT}} X(k)$$

$$x(n + N) = x(n) \quad \text{for all } n$$

$$X(k + N) = X(k) \quad \text{for all } k$$

PROPERTIES OF DFT

Linearity

$$x_1(n) \xleftrightarrow[N]{\text{DFT}} X_1(k)$$

$$x_2(n) \xleftrightarrow[N]{\text{DFT}} X_2(k)$$

$$a_1 x_1(n) + a_2 x_2(n) \xleftrightarrow[N]{\text{DFT}} a_1 X_1(k) + a_2 X_2(k)$$

PROPERTIES OF DFT

Circular Convolution

$$x_1(n) \xleftrightarrow[N]{\text{DFT}} X_1(k)$$

$$x_2(n) \xleftrightarrow[N]{\text{DFT}} X_2(k)$$

$$x_1(n)x_2(n) \xleftrightarrow[N]{\text{DFT}} \frac{1}{N} X_1(k) \circledast X_2(k)$$

$$x_1(n) \circledast x_2(n) \xleftrightarrow[N]{\text{DFT}} X_1(k)X_2(k)$$

PROPERTIES OF DFT

Time reversal

$$x(n) \xleftrightarrow[N]{\text{DFT}} X(k)$$

$$x((-n))_N = x(N - n) \xleftrightarrow[N]{\text{DFT}} X((-k))_N = X(N - k)$$

If the sequence is circularly folded , its DFT is also circularly folded.

PROPERTIES OF DFT

Time shift

$$x(n) \xleftrightarrow[N]{\text{DFT}} X(k)$$

$$x((n-l))_N \xleftrightarrow[N]{\text{DFT}} X(k)e^{-j2\pi kl/N}$$

PROPERTIES OF DFT

Conjugation

DFT of complex conjugate of any sequence is equal to complex conjugate of DFT of that sequence , with sequence delayed by k samples in frequency domain.

$$\text{DFT}\{x(n)\}=X(k)$$

$$\text{DFT}\{X^*(n)\}=X^*(N-k)$$

2D DISCRETE FOURIER TRANSFORM

The 2D-DFT of a rectangular image $f(m, n)$ of size $M \times N$ is represented as $F(k, l)$

$$f(m, n) \xrightarrow{\text{2D-DFT}} F(k, l)$$

where $F(k, l)$ is defined as

$$F(k, l) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) e^{-j\frac{2\pi}{M}mk} e^{-j\frac{2\pi}{N}nl}$$

For a square image $f(m, n)$ of size $N \times N$, the 2D DFT is defined as

$$F(k, l) = \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} f(m, n) e^{-j\frac{2\pi}{N}mk} e^{-j\frac{2\pi}{N}nl}$$

The inverse 2D Discrete Fourier Transform is given by

$$f(m, n) = \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} F(k, l) e^{j\frac{2\pi}{N}mk} e^{j\frac{2\pi}{N}nl}$$

The Fourier transform $F(k, l)$ is given by

$$F(k, l) = R(k, l) + jI(k, l)$$

where $R(k, l)$ represents the real part of the spectrum and $I(k, l)$ represents the imaginary part

TWIDDLE FACTOR

Phase or Twiddle Factor

By the definition of DFT, the N-point DFT is given by,

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{\frac{-j2\pi nk}{N}} ; \text{ for } k = 0, 1, 2, \dots, N-1 \quad \dots(5.24)$$

To simplify the notation it is desirable to define the complex valued phase factor W_N (also called as twiddle factor) which is an N^{th} root of unity as,

$$W_N = e^{\frac{-j2\pi}{N}}$$

Here, W represents a complex number $1 \angle -2\pi$. Hence the phase or argument of W is -2π . Therefore, when a number is multiplied by W , only its phase changes by -2π but magnitude remains same.

$$\therefore W = e^{-j2\pi}$$

The phase value -2π of W can be multiplied by any integer and it is represented as prefix in W . For example multiplying -2π by k can be represented as W^k .

$$\therefore e^{-j2\pi \times k} \Rightarrow W^k$$

TWIDDLE FACTOR

The phase value -2π of W can be divided by any integer and it is represented as suffix in W . For example dividing -2π by N can be represented as W_N .

$$\begin{aligned} \therefore e^{-j2\pi \times \frac{1}{N}} &= e^{-j2\pi \times \frac{1}{N}} \Rightarrow W_N \\ \therefore e^{\frac{-j2\pi nk}{N}} &= \left(e^{-j2\pi}\right)^{\frac{nk}{N}} = W_N^{nk} \end{aligned} \quad \text{.....(5.25)}$$

Using equation (5.25) the equation (5.24) can be written as,

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad ; \text{ for } k = 0, 1, 2, \dots, N-1 \quad \text{.....(5.26)}$$

The equation (5.26) is the definition of N -point DFT using phase factor, and this equation is popularly used in FFT.

TWIDDLE FACTOR

$$\text{DFT: } X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}, \quad k = 0, 1, 2, \dots, N-1$$

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}, \quad k = 0, 1, \dots, N-1$$

$$\text{IDFT: } x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}, \quad n = 0, 1, 2, \dots, N-1$$

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-kn}, \quad n = 0, 1, \dots, N-1$$

DFT:

$$X(k) := \sum_{n=0}^{N-1} \left[x(n) (W_N)^{nk} \right]$$

For N of length 4, range of $n, k = [0 \ 1 \ 2 \ 3]$ each.

$$\text{Hence } X(n) = x(0)W_N^{n.0} + x(1)W_N^{n.1} + x(2)W_N^{n.2} + x(3)W_N^{n.3}$$

		$x(0)$	$x(1)$	$x(2)$	$x(3)$
$X(0)$	=	$W_4^{0 \times 0}$	$W_4^{0 \times 1}$	$W_4^{0 \times 2}$	$W_4^{0 \times 3}$
$X(1)$	=	$W_4^{1 \times 0}$	$W_4^{1 \times 1}$	$W_4^{1 \times 2}$	$W_4^{1 \times 3}$
$X(2)$	=	$W_4^{2 \times 0}$	$W_4^{2 \times 1}$	$W_4^{2 \times 2}$	$W_4^{2 \times 3}$
$X(3)$	=	$W_4^{3 \times 0}$	$W_4^{3 \times 1}$	$W_4^{3 \times 2}$	$W_4^{3 \times 3}$

		x(0)	x(1)	x(2)	x(3)
X(0)	=	$W_4^{0 \times 0}$	$W_4^{0 \times 1}$	$W_4^{0 \times 2}$	$W_4^{0 \times 3}$
X(1)	=	$W_4^{1 \times 0}$	$W_4^{1 \times 1}$	$W_4^{1 \times 2}$	$W_4^{1 \times 3}$
X(2)	=	$W_4^{2 \times 0}$	$W_4^{2 \times 1}$	$W_4^{2 \times 2}$	$W_4^{2 \times 3}$
X(3)	=	$W_4^{3 \times 0}$	$W_4^{3 \times 1}$	$W_4^{3 \times 2}$	$W_4^{3 \times 3}$

Can be rewritten as:

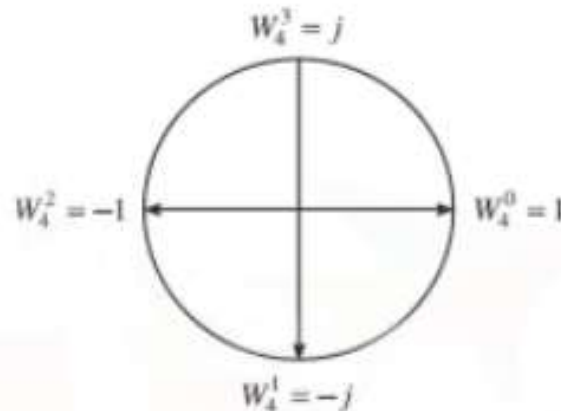
		x(0)	x(1)	x(2)	x(3)
X(0)	=	W_4^0	W_4^0	W_4^0	W_4^0
X(1)	=	W_4^0	W_4^1	W_4^2	W_4^3
X(2)	=	W_4^0	W_4^2	W_4^4	W_4^6
X(3)	=	W_4^0	W_4^3	W_4^6	W_4^9

CYCLIC PROPERTY OF TWIDDLE FACTORS

The "cyclic property" of twiddle factors means that their values repeat periodically, meaning that the sequence of twiddle factors for a given FFT size will repeat after a certain number of cycles, which is equal to the size of the DFT (N) itself; essentially, the values "wrap around" and start repeating again, allowing for efficient computation in FFT algorithms by reusing ca

For a 4-point DFT

$$W_4^0 = 1 \quad W_4^1 = -j \quad W_4^2 = -1 \quad W_4^3 = j$$



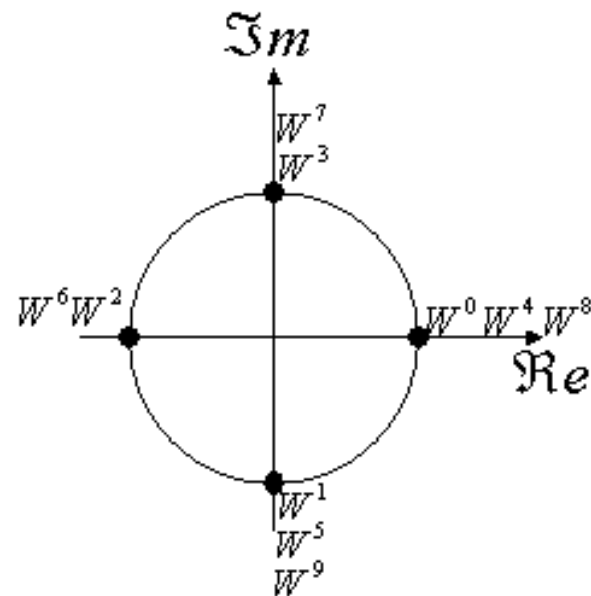
(a) 4-point DFT

$$W = e^{-j\left(\frac{2\pi}{4}\right)}$$

$$X(k) = \sum_{n=0}^3 x(n) \cdot W^{nk} \quad (k = 0, 1, 2, 3)$$



$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} W^0 & W^0 & W^0 & W^0 \\ W^0 & W^1 & W^2 & W^3 \\ W^0 & W^2 & W^4 & W^6 \\ W^0 & W^3 & W^6 & W^9 \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$



Compute the DFT of the four-point sequence $x(n)=\{0,1,2,3\}$

$$W_4 = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^0 & W_4^2 \\ 1 & W_4^3 & W_4^2 & W_4^1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$X_4 = W_4 x_4 = \begin{bmatrix} 6 \\ -2 + 2j \\ -2 \\ -2 - 2j \end{bmatrix}$$

$$\begin{pmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{pmatrix} := \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{pmatrix} \cdot \begin{pmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{pmatrix}$$

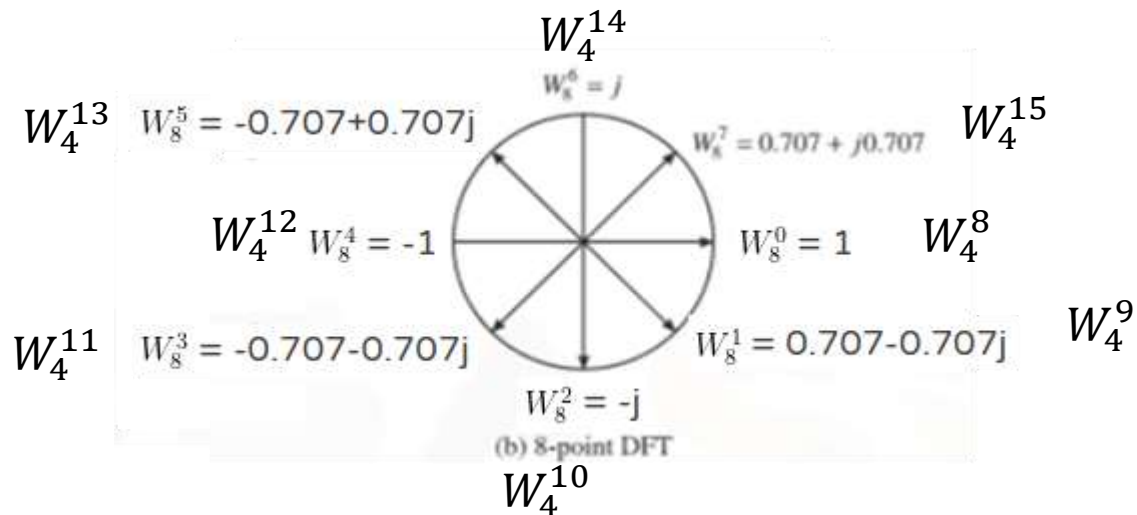
The effective determinant of above is $1/4$.
 Conversion in system matrices of DFT and IDFT obtained by replacing j by $-j$.
 Inversion of above matrix is:

$$\begin{pmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{pmatrix} := \left[\frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & j & -1 & -j \\ 1 & -1 & 1 & -1 \\ 1 & -j & -1 & j \end{pmatrix} \cdot \begin{pmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{pmatrix} \right]$$

CYCLIC PROPERTY OF TWIDDLE FACTORS

For an 8-point DFT

$$\begin{aligned}
 W_8^0 &= 1 & W_8^1 &= 0.707 - 0.707j & W_8^2 &= -j & W_8^3 &= -0.707 - 0.707j & W_8^4 &= -1 \\
 W_8^5 &= -0.707 + 0.707j & W_8^6 &= j & W_8^7 &= 0.707 + 0.707j
 \end{aligned}$$



FOR 8 POINT DFT: N=8

The system Matrix is:

$W_8^{0 \times 0}$	$W_8^{0 \times 1}$	$W_8^{0 \times 2}$	$W_8^{0 \times 3}$	$W_8^{0 \times 4}$	$W_8^{0 \times 5}$	$W_8^{0 \times 6}$	$W_8^{0 \times 7}$
$W_8^{1 \times 0}$	$W_8^{1 \times 1}$	$W_8^{1 \times 2}$	$W_8^{1 \times 3}$	$W_8^{1 \times 4}$	$W_8^{1 \times 5}$	$W_8^{1 \times 6}$	$W_8^{1 \times 7}$
$W_8^{2 \times 0}$	$W_8^{2 \times 1}$	$W_8^{2 \times 2}$	$W_8^{2 \times 3}$	$W_8^{2 \times 4}$	$W_8^{2 \times 5}$	$W_8^{2 \times 6}$	$W_8^{2 \times 7}$
$W_8^{3 \times 0}$	$W_8^{3 \times 1}$	$W_8^{3 \times 2}$	$W_8^{3 \times 3}$	$W_8^{3 \times 4}$	$W_8^{3 \times 5}$	$W_8^{3 \times 6}$	$W_8^{3 \times 7}$
$W_8^{4 \times 0}$	$W_8^{4 \times 1}$	$W_8^{4 \times 2}$	$W_8^{4 \times 3}$	$W_8^{4 \times 4}$	$W_8^{4 \times 5}$	$W_8^{4 \times 6}$	$W_8^{4 \times 7}$
$W_8^{5 \times 0}$	$W_8^{5 \times 1}$	$W_8^{5 \times 2}$	$W_8^{5 \times 3}$	$W_8^{5 \times 4}$	$W_8^{5 \times 5}$	$W_8^{5 \times 6}$	$W_8^{5 \times 7}$
$W_8^{6 \times 0}$	$W_8^{6 \times 1}$	$W_8^{6 \times 2}$	$W_8^{6 \times 3}$	$W_8^{6 \times 4}$	$W_8^{6 \times 5}$	$W_8^{6 \times 6}$	$W_8^{6 \times 7}$
$W_8^{7 \times 0}$	$W_8^{7 \times 1}$	$W_8^{7 \times 2}$	$W_8^{7 \times 3}$	$W_8^{7 \times 4}$	$W_8^{7 \times 5}$	$W_8^{7 \times 6}$	$W_8^{7 \times 7}$

THE CHARACTERISTIC MATRIX OF 8-POINT DFT IS:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \frac{1-j}{\sqrt{2}} & -j & \frac{-(1+j)}{\sqrt{2}} & -1 & \frac{-(1-j)}{\sqrt{2}} & j & \frac{1+j}{\sqrt{2}} \\ 1 & -j & -1 & j & 1 & -j & -1 & j \\ 1 & \frac{-(1+j)}{\sqrt{2}} & j & \frac{1-j}{\sqrt{2}} & -1 & \frac{1+j}{\sqrt{2}} & -j & \frac{-(1-j)}{\sqrt{2}} \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & \frac{-(1-j)}{\sqrt{2}} & -j & \frac{1+j}{\sqrt{2}} & -1 & \frac{1-j}{\sqrt{2}} & j & \frac{-(1+j)}{\sqrt{2}} \\ 1 & j & -1 & -j & 1 & j & -1 & -j \\ 1 & \frac{1+j}{\sqrt{2}} & j & \frac{-(1-j)}{\sqrt{2}} & -1 & \frac{-(1+j)}{\sqrt{2}} & -j & \frac{(1-j)}{\sqrt{2}} \end{bmatrix}$$

DFT AND ITS INVERSE

CONVERSION IN SYSTEM MATRICES OF DFT AND IDFT OBTAINED BY REPLACING j BY $-j$.

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \frac{1-j}{\sqrt{2}} & -j & \frac{-(1+j)}{\sqrt{2}} & -1 & \frac{-(1-j)}{\sqrt{2}} & j & \frac{1+j}{\sqrt{2}} \\ 1 & -j & -1 & j & 1 & -j & -1 & j \\ 1 & \frac{-(1+j)}{\sqrt{2}} & j & \frac{1-j}{\sqrt{2}} & -1 & \frac{1+j}{\sqrt{2}} & -j & \frac{-(1-j)}{\sqrt{2}} \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & \frac{-(1-j)}{\sqrt{2}} & -j & \frac{1+j}{\sqrt{2}} & -1 & \frac{1-j}{\sqrt{2}} & j & \frac{-(1+j)}{\sqrt{2}} \\ 1 & j & -1 & -j & 1 & j & -1 & -j \\ 1 & \frac{1+j}{\sqrt{2}} & j & \frac{-(1-j)}{\sqrt{2}} & -1 & \frac{-(1+j)}{\sqrt{2}} & -j & \frac{(1-j)}{\sqrt{2}} \end{bmatrix}$$

DFT

$$\frac{1}{8} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \frac{1+j}{\sqrt{2}} & j & -\frac{1-j}{\sqrt{2}} & -1 & -\frac{1+j}{\sqrt{2}} & -j & \frac{1-j}{\sqrt{2}} \\ 1 & j & -1 & -j & 1 & j & -1 & -j \\ 1 & -\frac{1-j}{\sqrt{2}} & -j & \frac{1+j}{\sqrt{2}} & -1 & \frac{1-j}{\sqrt{2}} & j & -\frac{1+j}{\sqrt{2}} \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & -\frac{1+j}{\sqrt{2}} & j & \frac{1-j}{\sqrt{2}} & -1 & \frac{1+j}{\sqrt{2}} & -j & -\frac{1-j}{\sqrt{2}} \\ 1 & -j & -1 & j & 1 & -j & -1 & j \\ 1 & \frac{1-j}{\sqrt{2}} & -j & -\frac{1+j}{\sqrt{2}} & -1 & -\frac{1-j}{\sqrt{2}} & j & \frac{1+j}{\sqrt{2}} \end{pmatrix}$$

Effective
determinant

IDFT

DFT OF IMAGE MATRIX (2D)

Example 4.4 Compute the 2D DFT of the 4×4 grayscale image given below.

$$f[m, n] = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Solution The 2D DFT of the image $f[m, n]$ is represented as $F[k, l]$.

$$F[k, l] = \text{kernel} \times f[m, n] \times (\text{kernel})^T$$

The kernel or basis of the Fourier transform for $N = 4$ is given by

The DFT basis for $N = 4$ is given by

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

DFT OF IMAGE MATRIX (2D)

$$F(k, l) = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$F(k, l) = \begin{bmatrix} 4 & 4 & 4 & 4 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} = \begin{bmatrix} 16 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Example 4.5 Compute the inverse 2D DFT of the transform coefficients given by

$$F[k, l] = \begin{bmatrix} 16 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Solution The inverse 2D DFT of the Fourier coefficients $F[k, l]$ is given by $f[m, n]$ as

$$f[m, n] = \frac{1}{N^2} \times \text{kernel} \times F[k, l] \times (\text{kernel})^T$$

In this example, $N = 4$

$$f[m, n] = \frac{1}{16} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} \times \begin{bmatrix} 16 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$f[m, n] = \frac{1}{16} \times \begin{bmatrix} 16 & 0 & 0 & 0 \\ 16 & 0 & 0 & 0 \\ 16 & 0 & 0 & 0 \\ 16 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$f[m, n] = \frac{1}{16} \times \begin{bmatrix} 16 & 16 & 16 & 16 \\ 16 & 16 & 16 & 16 \\ 16 & 16 & 16 & 16 \\ 16 & 16 & 16 & 16 \end{bmatrix}$$

$$f[m, n] = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

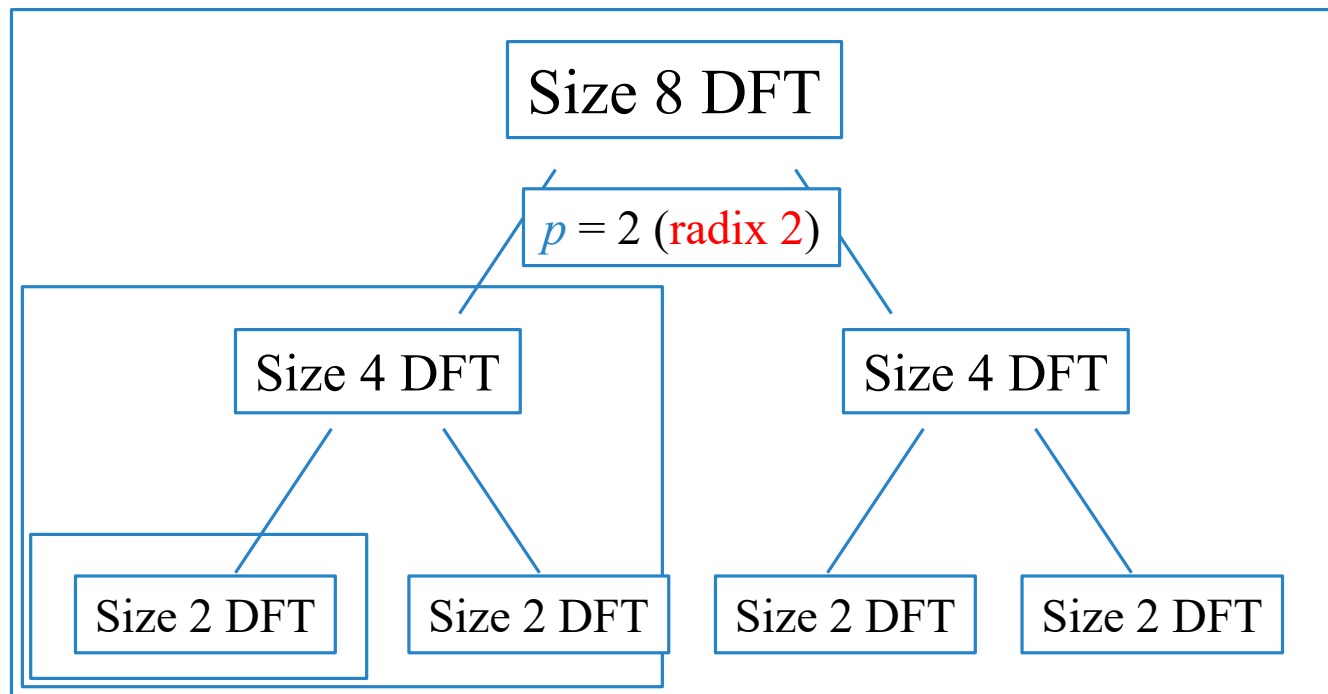
FAST FOURIER TRANSFORM

FFT is method of computing DFT with reduced number of calculations.

Computational efficiency is achieved if we adopt a divide and conquer approach.

Approach is based on decomposition of N point DFT into successively smaller DFTs.

FFT-DIVIDE & CONQUER



RADIX-2 FFT

Radix-x: here 'x' represents number of samples in each group made at the first stage. They are generally equal.

$$N=2^m$$

m is number of stages

N point sequence is decimated into 2 point sequences, further from the results of 2 point DFTs, the 4 point DFTs can be computed and so on.

RADIX -2

Radix-2 is the first FFT algorithm. It was proposed by Cooley and Tukey in 1965.

The algorithms appear either in

- (a) **D**ecimation **I**n **T**ime (DIT),
- (b) **D**ecimation **I**n **F**requency (DIF).

DIT and DIF, both yield same complexity and results. They are complementary.

We shall stress on 8 to radix 2 DIT FFT

Number of Calculations in N-point DFT

Let, $X(k)$ be N-point DFT of an L-point discrete time sequence $x(n)$, where $N \geq L$. Now, the N-point DFT is a sequence consisting of N-complex numbers. Each complex number of the sequence is calculated using the following equation (equation 5.2).

$$\begin{aligned}
 X(k) &= \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N} ; \text{ for } k = 0, 1, 2, \dots, N-1 \\
 &= x(0) e^0 + x(1) e^{-j2\pi k/N} + x(2) e^{-j4\pi k/N} + x(3) e^{-j6\pi k/N} + \dots + x(N-1) e^{-j2(N-1)\pi k/N} \\
 \therefore X(k) &= \underbrace{x(0) e^0}_{\text{Complex multiplication}} + \underbrace{x(1) e^{-j2\pi k/N}}_{\text{Complex multiplication}} + \underbrace{x(2) e^{-j4\pi k/N}}_{\text{Complex multiplication}} + \underbrace{x(3) e^{-j6\pi k/N}}_{\text{Complex multiplication}} + \dots + \underbrace{x(N-1) e^{-j2(N-1)\pi k/N}}_{\text{Complex multiplication}} \\
 &\quad \underbrace{\hspace{10em}}_{N-1 \text{ Complex additions}}
 \end{aligned}$$

The number of calculations to calculate $X(k)$ for one value of k are,

N number of complex multiplications and

N - 1 number of complex additions.

The $X(k)$ is a sequence consisting of N complex numbers.

Therefore, the number of calculations to calculate all the N complex numbers of the $X(k)$ are,

$N \times N = N^2$ number of complex multiplications and

$N \times (N - 1) = N(N - 1)$ number of complex additions.

Hence, in direct computation of N-point DFT, the total number of complex additions are $N(N - 1)$ and total number of complex multiplications are N^2 .

Number of Calculations in Radix-2 FFT

In radix-2 FFT, $N = 2^m$, and so there will be m stages of computations, where $m = \log_2 N$, with each stage having $N/2$ butterflies. (Refer section 5.7.2 and 5.8.2).

The number of calculations in one butterfly are,
1 number of Complex multiplication and
2 number of Complex additions.

There are $\frac{N}{2}$ butterflies in each stage.

Therefore, number of calculations in one stage are,

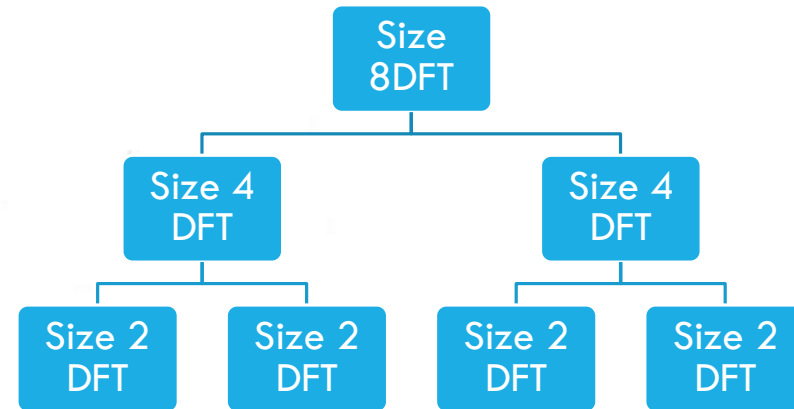
$$\frac{N}{2} \times 1 = \frac{N}{2} \text{ complex multiplications and}$$

$$\frac{N}{2} \times 2 = N \text{ complex additions.}$$

The N -point DFT involves m stages of computations. Therefore, the number of calculations for m stages are,

$$m \times \frac{N}{2} = \log_2 N \times \frac{N}{2} = \frac{N}{2} \log_2 N \text{ complex multiplications and}$$

$$m \times N = \log_2 N \times N = N \log_2 N \text{ complex additions.}$$



Number of points N	Direct Computation		Radix-2 FFT	
	Complex additions $N(N-1)$	Complex Multiplications N^2	Complex additions $N \log_2 N$	Complex Multiplications $(N/2) \log_2 N$
$4 (= 2^2)$	12	16	$4 \times \log_2 2^2 = 4 \times 2 = 8$	$\frac{4}{2} \times \log_2 2^2 = \frac{4}{2} \times 2 = 4$
$8 (= 2^3)$	56	64	$8 \times \log_2 2^3 = 8 \times 3 = 24$	$\frac{8}{2} \times \log_2 2^3 = \frac{8}{2} \times 3 = 12$
$16 (= 2^4)$	240	256	$16 \times \log_2 2^4 = 16 \times 4 = 64$	$\frac{16}{2} \times \log_2 2^4 = \frac{16}{2} \times 4 = 32$
$32 (= 2^5)$	992	1,024	$32 \times \log_2 2^5 = 32 \times 5 = 160$	$\frac{32}{2} \times \log_2 2^5 = \frac{32}{2} \times 5 = 80$
$64 (= 2^6)$	4,032	4,096	$64 \times \log_2 2^6 = 64 \times 6 = 384$	$\frac{64}{2} \times \log_2 2^6 = \frac{64}{2} \times 6 = 192$
$128 (= 2^7)$	16,256	16,384	$128 \times \log_2 2^7 = 128 \times 7 = 896$	$\frac{128}{2} \times \log_2 2^7 = \frac{128}{2} \times 7 = 448$

DIT- FFT FLOWGRAPHS

The **Radix-2 Decimation in Time (DIT) Fast Fourier Transform (FFT)** is an efficient algorithm to compute the **Discrete Fourier Transform (DFT)**. It reduces the computational complexity from $O(N^2)$ (in direct DFT computation) to $O(N \log_2 N)$.

Steps to compute the **DFT using Radix-2 DIT FFT**:

Step 1: Check for Radix-2 Condition

- Ensure that the number of input points, **N**, is a power of 2 ($N = 2^m$).
- If **N** is not a power of 2, pad the sequence with zeros until it becomes a power of 2.

DIT- FFT FLOWGRAPHS

Step 2: Bit-Reversal Permutation

- Rearrange the input sequence $x(n)$ into bit-reversed order.
- The index positions of the input sequence are converted into their binary equivalents, then reversed, and finally mapped to the new sequence.

Step 3: Perform FFT Computation Using Butterfly Operations

- The algorithm follows a **divide-and-conquer** approach by recursively breaking down the DFT into smaller computations.
- The number of **stages** in the FFT is $\log_2 N$.
- Each stage consists of multiple **butterfly computations**, where a butterfly operation processes two inputs to produce two outputs.

For each stage s (where $s = 1, 2, \dots, \log_2 N$):

1. Divide the sequence into $N/2$ pairs for butterfly computations.
2. Compute the **twiddle factors**:

$$W_N^k = e^{-j(2\pi k/N)}$$

3. Update the sequence using the butterfly equations:

$$X_{\text{even}} = A + W_N^k B$$

$$X_{\text{odd}} = A - W_N^k B$$

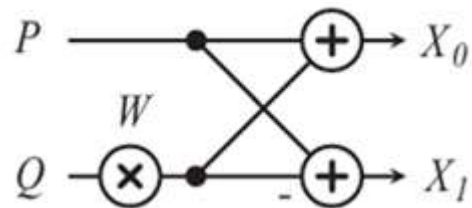
where **A** and **B** are the inputs to the butterfly operation.

Step 4: Iterate for All Stages

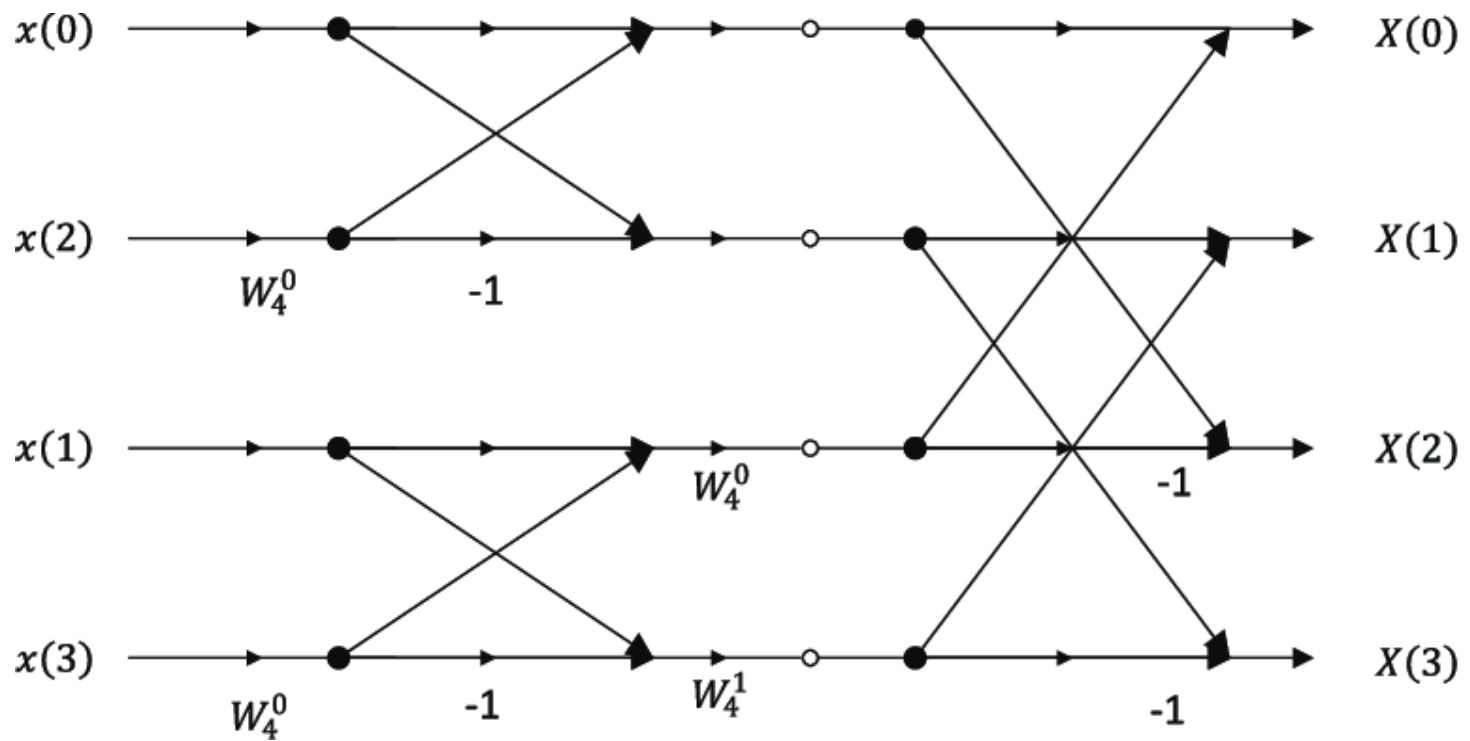
- Repeat the butterfly computations for $\log_2 N$ stages.
- The number of groups (or sub-DFTs) doubles at each stage, while the number of points per group halves.

Step 5: Output the Final FFT Coefficients

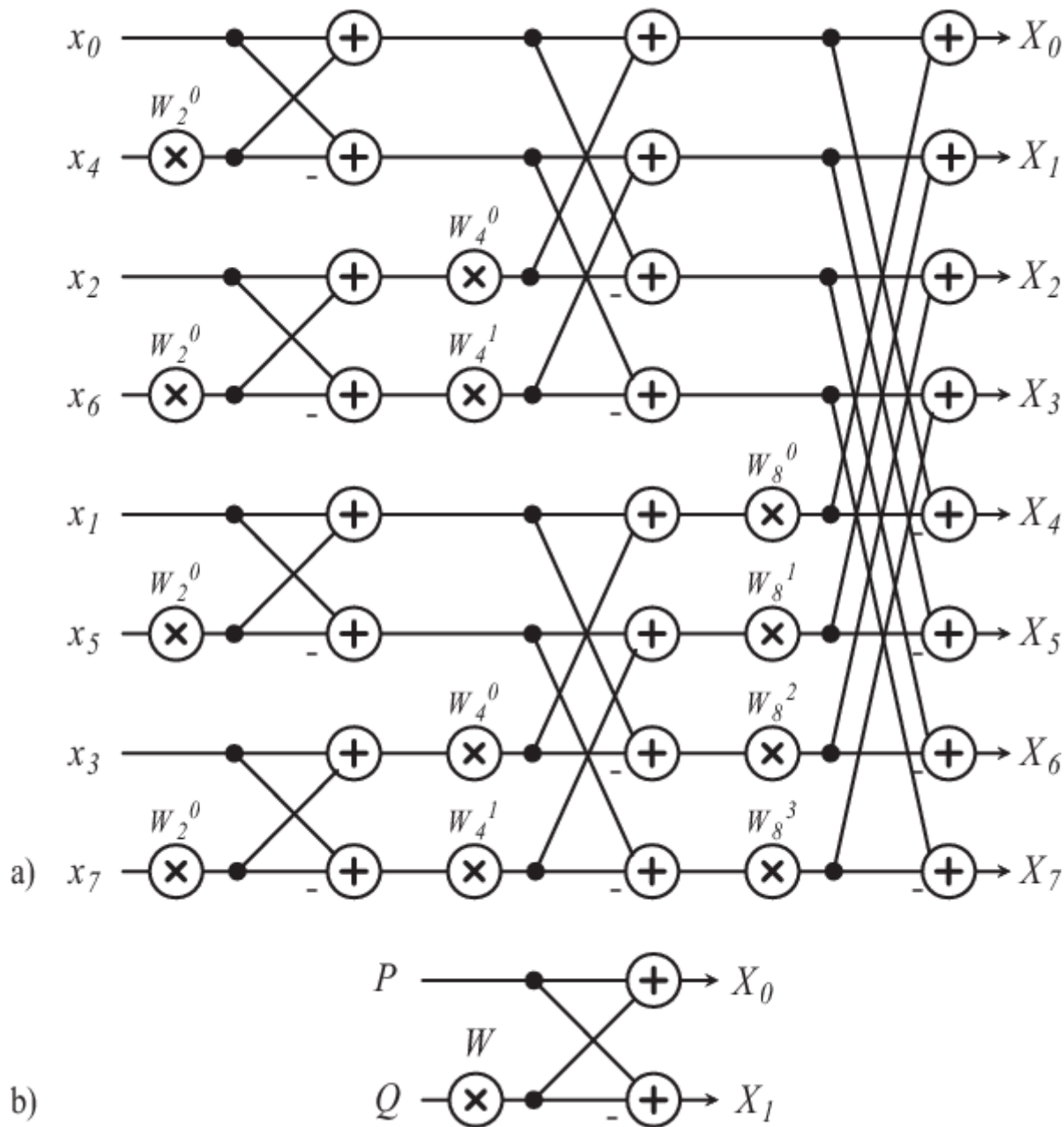
- After completing $\log_2 N$ stages, the transformed sequence $X(k)$ represents the DFT of the input sequence $x(n)$.



radix-2 DIT butterfly operation.



radix-4 DIT butterfly operation.



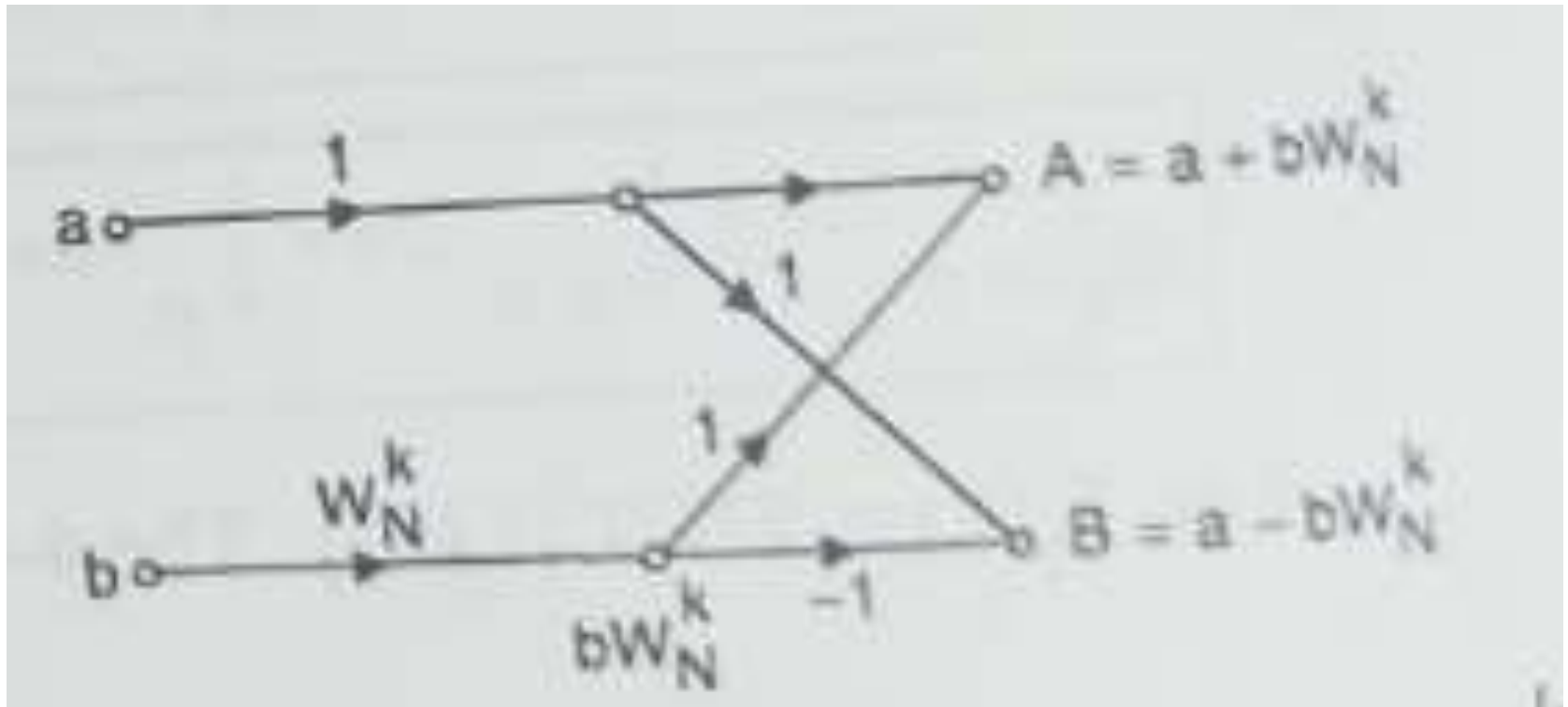
(a) Signal flow graph of 8-point radix-2 DIT FFT (b) radix-2 DIT butterfly operation.



Refer example 5.5, page 5.38 of digital signal processing,
Nagoorkani

DIT-FFT

Steps to find DFT using Radix 2 DIT FFT:



Ex: Draw the signal flow graph of the 4-point FFT, then use it to find the spectrum of the sequence $x(n)=\{1,-1,2,3\}$.

The even numbered samples are $x(0)$ and $x(2)$

The odd numbered samples are $x(1)$ and $x(3)$

$$\underline{X(k)} = G(k) + W_4^k H(k) \quad 0 \leq k \leq 1$$

$$X(0) = G(0) + W_4^0 H(0)$$

$$X(1) = G(1) + W_4^1 H(1)$$

$$X(2) = G(0) + W_4^2 H(0)$$

$$X(3) = G(3) + W_4^3 H(3)$$

And again $G(3)=G(1)$ and $H(3)=H(1)$, then:

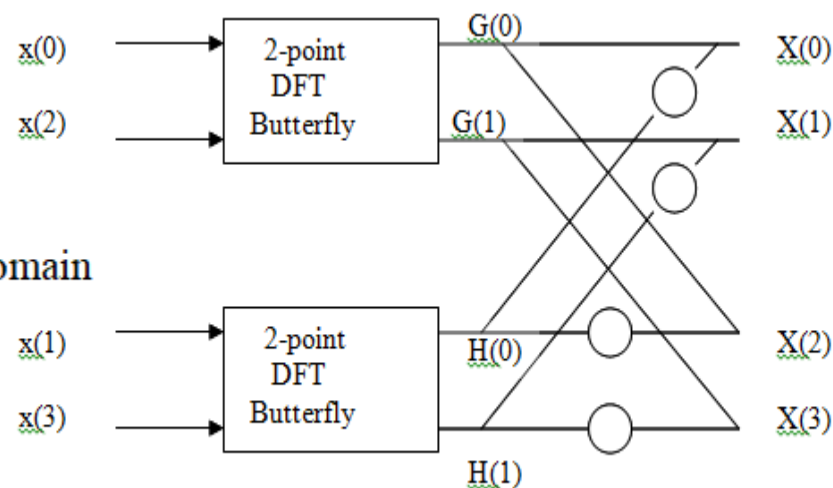
$$X(3) = G(1) + W_4^3 H(1)$$

$$E = \sum x^2(n) = 1+4+1+9=15 = \text{energy from time domain}$$

$$\frac{1}{N} \sum |X(k)|^2 = (25+17+1+17)/4 = 15 = \text{energy from frequency domain}$$

decimal	binary	binary	decimal
0	00	00	0
1	01	10	2
2	10	01	1
3	11	11	3

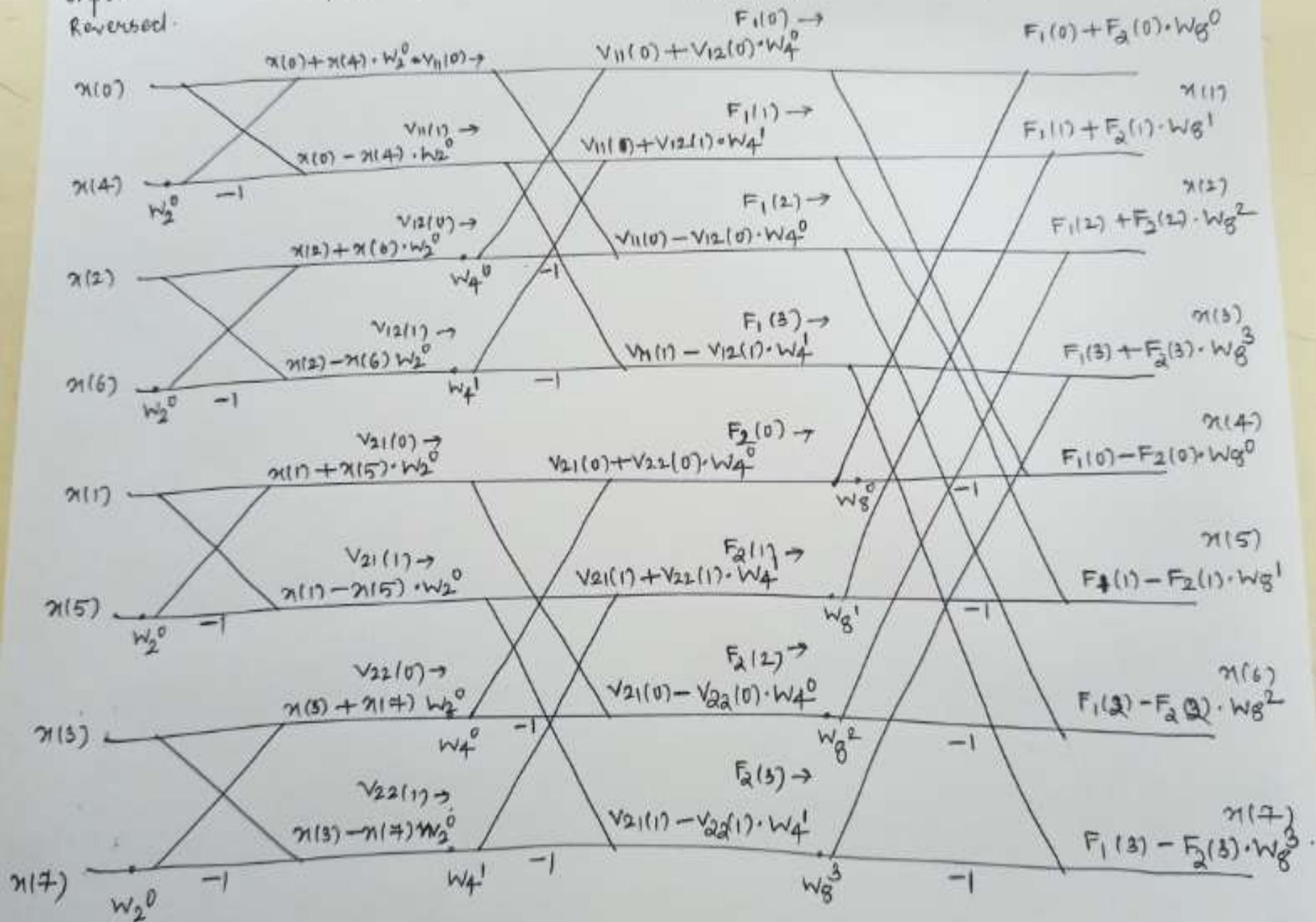
↑
Mirror



Input
Reversed.

8 point $\rightarrow$ DIT-FFT (Butterfly diagram for sequence)

Output
 $x(0)$



2 point

4 point

8

$$W_2^0 = 1$$

$$W_4^0 = 1$$

$$W_4^1 = -j$$

$$W_N^k = e^{-j \frac{2\pi k}{N}}$$

$$W_2^0 = e^0 = 1$$

$$W_4^0 = e^0 = 1$$

$$W_4^1 = \frac{e^{-j \frac{2\pi (1)}{4}}}{e^{-j \frac{\pi}{2}}}$$

$$\cos \pi/2 - j \sin \pi/2$$

4 point

8 point

$$W_4^0 = 1$$

$$W_8^0 = 1$$

$$W_4^1 = -j$$

$$W_8^1 = \frac{1}{\sqrt{2}} - j \frac{1}{\sqrt{2}}$$

$$W_8^2 = -j$$

$$W_8^3 = -\frac{1}{\sqrt{2}} - j \frac{1}{\sqrt{2}}$$

DISCRETE COSINE TRANSFORM (DCT)

The discrete cosine transforms are the members of a family of real-valued discrete sinusoidal unitary transforms.

A discrete cosine transform consists of a set of basis vectors that are sampled cosine functions.

DCT is a technique for converting a signal into elementary frequency components and it is widely used in image compression.

Thus, the kernel of a one-dimensional discrete cosine transform is given by

$$X[k] = \alpha(k) \sum_{n=0}^{N-1} x[n] \cos \left\{ \frac{(2n+1)\pi k}{2N} \right\}, \text{ where } 0 \leq k \leq N-1$$

$$\alpha(k) = \sqrt{\frac{1}{N}} \text{ if } k = 0$$

$$\alpha(k) = \sqrt{\frac{2}{N}} \text{ if } k \neq 0$$

Example 4.10 Compute the discrete cosine transform (DCT) matrix for $N = 4$.

Solution The formula to compute the DCT matrix is given by

$$X[k] = \alpha(k) \sum_{n=0}^{N-1} x[n] \cos \left\{ \frac{(2n+1)\pi k}{2N} \right\}, \text{ where } 0 \leq k \leq N-1$$

where

$$\alpha(k) = \sqrt{\frac{1}{N}} \text{ if } k = 0$$

$$\alpha(k) = \sqrt{\frac{2}{N}} \text{ if } k \neq 0$$

In our case, the value of $N = 4$. Substituting $N = 4$ in the expression of $X[k]$ we get

$$X[k] = \alpha(k) \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1)\pi k}{8} \right]$$

Substituting $k = 0$ in Eq. (4.82), we get

$$\begin{aligned} X[0] &= \sqrt{\frac{1}{4}} \times \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1)\pi \times 0}{8} \right] \\ &= \frac{1}{2} \times \sum_{n=0}^3 x[n] \cos(0) = \frac{1}{2} \times \sum_{n=0}^3 x[n] \times 1 \\ &= \frac{1}{2} \times \sum_{n=0}^3 x[n] \\ &= \frac{1}{2} \times \{x(0) + x(1) + x(2) + x(3)\} \\ X[0] &= \frac{1}{2}x(0) + \frac{1}{2}x(1) + \frac{1}{2}x(2) + \frac{1}{2}x(3) \end{aligned}$$

Substituting $k = 1$ in Eq. (4.82), we get

$$\begin{aligned}X[1] &= \sqrt{\frac{2}{4}} \times \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1)\pi \times 1}{8} \right] \\&= \sqrt{\frac{1}{2}} \times \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1)\pi}{8} \right] \\&= \sqrt{\frac{1}{2}} \times \left\{ x(0) \times \cos \left(\frac{\pi}{8} \right) + x(1) \times \cos \left(\frac{3\pi}{8} \right) + x(2) \times \cos \left(\frac{5\pi}{8} \right) + x(3) \times \cos \left(\frac{7\pi}{8} \right) \right\} \\&= 0.707 \times \{ x(0) \times 0.9239 + x(1) \times 0.3827 + x(2) \times (-0.3827) + x(3) \times (-0.9239) \} \\X(1) &= 0.6532x(0) + 0.2706x(1) - 0.2706x(2) - 0.6532x(3)\end{aligned}$$

Substituting $k = 2$ in Eq. (4.82), we get

$$\begin{aligned} X(2) &= \sqrt{\frac{2}{4}} \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1)\pi \times 2}{8} \right] \\ &= \sqrt{\frac{1}{2}} \times \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1)\pi}{4} \right] \\ &= \sqrt{\frac{1}{2}} \times \left\{ x(0) \times \cos \left(\frac{\pi}{4} \right) + x(1) \times \cos \left(\frac{3\pi}{4} \right) + x(2) \times \cos \left(\frac{5\pi}{4} \right) + x(3) \times \cos \left(\frac{7\pi}{4} \right) \right\} \\ &= 0.7071 \times \{ x(0) \times 0.7071 + x(1) \times (-0.7071) + x(2) \times (-0.7071) + x(3) \times (0.7071) \} \\ X(2) &= 0.5x(0) - 0.5x(1) - 0.5x(2) + 0.5x(3) \end{aligned}$$

Substituting $k = 3$ in Eq. (4.82), we get

$$\begin{aligned} X(3) &= \sqrt{\frac{2}{4}} \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1)\pi \times 3}{8} \right] \\ &= \sqrt{\frac{1}{2}} \times \sum_{n=0}^3 x[n] \cos \left[\frac{(2n+1) \times 3\pi}{8} \right] \\ &= \sqrt{\frac{1}{2}} \times \left\{ x(0) \times \cos \left(\frac{3\pi}{8} \right) + x(1) \times \cos \left(\frac{9\pi}{8} \right) + x(2) \times \cos \left(\frac{15\pi}{8} \right) + x(3) \times \cos \left(\frac{21\pi}{8} \right) \right\} \\ &= 0.7071 \times \{ x(0) \times 0.3827 + x(1) \times (-0.9239) + x(2) \times (0.9239) + x(3) \times (-0.3827) \} \end{aligned}$$

$$X(3) = 0.2706x(0) - 0.6533x(1) + 0.6533x(2) - 0.2706x(3)$$

Collecting the coefficients of $x(0)$, $x(1)$, $x(2)$ and $x(3)$ from $X(0)$, $X(1)$, $X(2)$ and $X(3)$, we get

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ X[3] \end{bmatrix} = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.6532 & 0.2706 & -0.2706 & -0.6532 \\ 0.5 & -0.5 & -0.5 & 0.5 \\ 0.2706 & -0.6533 & 0.6533 & -0.2706 \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ x[3] \end{bmatrix}$$

IDCT

$$x[n] = \alpha(k) \sum_{k=0}^{N-1} X[k] \cos \left[\frac{(2n+1)\pi k}{2N} \right], 0 \leq n \leq N-1$$

SYMMETRIC AND ASYMMETRIC

Transformation Matrix – T

Image Matrix – f

If Transformation matrix is symmetric:

$$F = T f T$$

If Transformation matrix is asymmetric:

$$F = T f T'$$

HADAMARD TRANSFORM

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

The Hadamard matrix of order $2N$ can be generated by Kronecker product operation:

$$H_{2N} = \begin{bmatrix} H_N & H_N \\ H_N & -H_N \end{bmatrix}$$

Substituting $N = 2$ in Eq. (4.59), we get

$$H_4 = \begin{bmatrix} H_2 & H_2 \\ H_2 & -H_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

Similarly, substituting $N = 4$ in Eq. (4.59), we get

$$H_8 = \begin{bmatrix} H_4 & H_4 \\ H_4 & -H_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{bmatrix}$$

HADAMARD TRANSFORM

Hadamard of Sequence

$$X[n] = \{H(N).x(n)\}$$

Inverse Hadamard of Sequence

$$x[n] = 1/N \{H(N).X(n)\}$$

Hadamard of image (2D)

Hadamard is symmetric transform

$$F = T f T$$

$$H_8 = \begin{bmatrix} H_4 & H_4 \\ H_4 & -H_4 \end{bmatrix} = \left[\begin{array}{cccccccc|c} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 & 7 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 3 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 4 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 & 6 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 & 2 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 & 5 \end{array} \right]$$



$$W(8) = \left[\begin{array}{cccccccc|c} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 & 2 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 & 3 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 & 4 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 & 5 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 & 6 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 & 7 \end{array} \right]$$

WALSH TRANSFORM

$$W(8) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix},$$

HAAR TRANSFORM

Haar Transform

- For $N=2$ dan $N=4$:

$$Hr_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$Hr_4 = \frac{1}{\sqrt{4}} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & -1 & -1 \\ \sqrt{2} & -\sqrt{2} & 0 & 0 \\ 0 & 0 & \sqrt{2} & -\sqrt{2} \end{bmatrix}$$

- Properties of Haar Transform:

1. Real and orthogonal: $Hr = Hr^*$ dan $Hr^{-1} = Hr^T$
2. Very fast transform : $O(N)$ operation on $N \times 1$ vector.
3. Poor energy compaction for images.

HAAR TRANSFORM

$$\mathbf{H}_3 = \frac{1}{\sqrt{8}} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ \sqrt{2} & \sqrt{2} & -\sqrt{2} & -\sqrt{2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \sqrt{2} & \sqrt{2} & -\sqrt{2} & -\sqrt{2} \\ 2 & -2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & -2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & -2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & -2 \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \\ h_{2,1} \\ h_{2,2} \\ h_{3,1} \\ h_{3,2} \\ h_{3,3} \\ h_{3,4} \end{bmatrix}$$

IMAGE ENHANCEMENT IN THE FREQUENCY DOMAIN

If we multiply each element of the Fourier coefficient by a suitably chosen weighting function then we can **accentuate certain frequency components and attenuate others**.

This **selective enhancement or suppression of frequency components** is termed Fourier filtering or frequency domain filtering.

The spatial representation of image data **describes the adjacency relationship between the pixels**.

On the other hand, the frequency domain **representation clusters the image data according to their frequency distribution**.

In frequency domain filtering, the **image data is dissected into various spectral bands**, where each band depicts a specific range of details within the image.

The **process of selective frequency inclusion or exclusion is termed frequency domain filtering**.

It is possible to perform filtering in the frequency domain by specifying the frequencies that should be kept and the frequencies that should be discarded.

IMAGE ENHANCEMENT IN THE FREQUENCY DOMAIN

Convolution in spatial domain = Multiplication in the frequency domain

Filtering in spatial domain = $f(m, n) * h(m, n)$

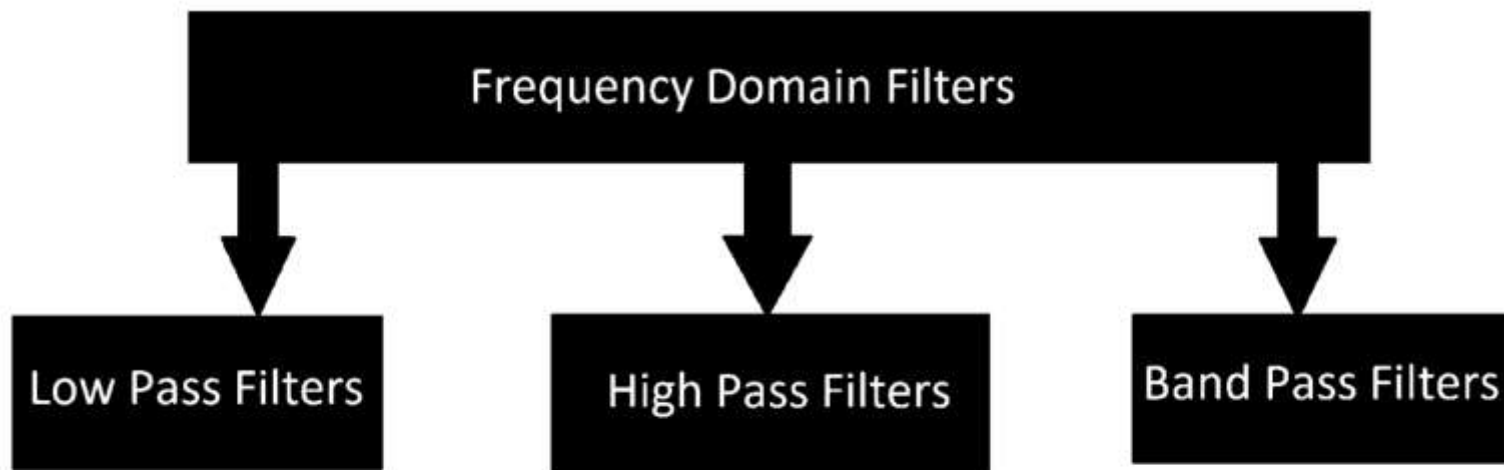
Filtering in the frequency domain = $F(k, l) \times H(k, l)$

$f(m, n)$ – original image

$F(k, l)$ – FT of original image

$h(m, n)$ – Filtering mask

$H(k, l)$ – FT of filtering mask



Classification of Frequency Domain Filters

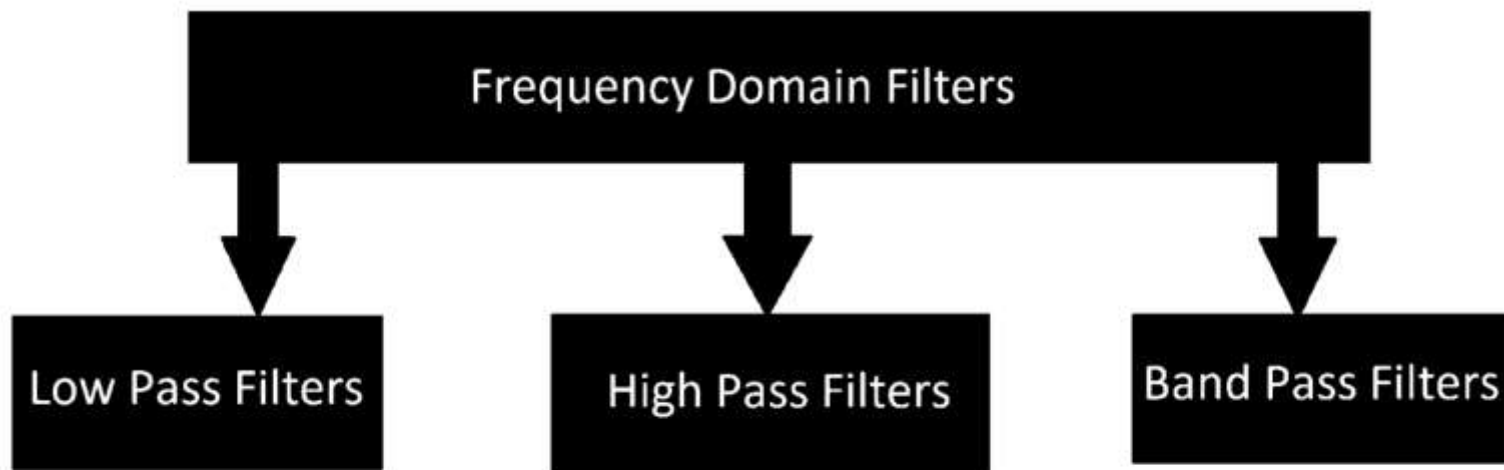
1. Low pass filter removes the high frequency components that means it keeps low frequency components. It is used for smoothing the image. It is used to smoothen the image by attenuating high frequency components and preserving low frequency components.

Mechanism of low pass filtering in frequency domain is given by:

$$G(u, v) = H(u, v) \cdot F(u, v)$$

where $F(u, v)$ is the Fourier Transform of original image

and $H(u, v)$ is the Fourier Transform of filtering mask



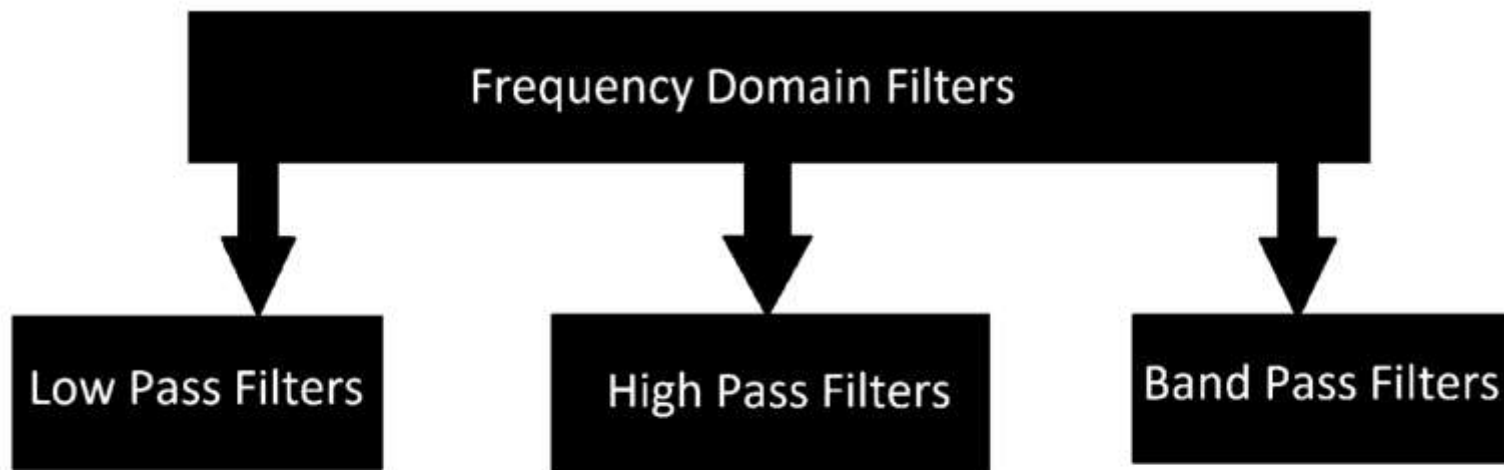
Classification of Frequency Domain Filters

2. High pass filter: High pass filter removes the low frequency components that means it keeps high frequency components. It is used for sharpening the image. It is used to sharpen the image by attenuating low frequency components and preserving high frequency components.

Mechanism of high pass filtering in frequency domain is given by:

$$H(u, v) = 1 - H'(u, v)$$

where $H(u, v)$ is the Fourier Transform of high pass filtering
and $H'(u, v)$ is the Fourier Transform of low pass filtering



Classification of Frequency Domain Filters

3. Band pass filter: Band pass filter removes the very low frequency and very high frequency components that means it keeps the moderate range band of frequencies. Band pass filtering is used to enhance edges while reducing the noise at the same time.

Band pass filter removes the very low frequency and very high frequency components that means it keeps the moderate range band of frequencies. Band pass filtering is used to enhance edges while reducing the noise at the same time.

LOW PASS (LP) FILTERS

Ideal low-pass filter (ILPF)

Butterworth low-pass filter (BLPF)

Gaussian low-pass filter (GLPF)

IDEAL LOW-PASS (LP) FILTERING

Preserves low frequencies, attenuates high frequencies.

Cut off all high-frequency components of the Fourier transform that are at a distance greater than a specified distance D_0 (cut off frequency) from the origin of the (centered) transform

The transfer function (frequency domain filter) is defined by

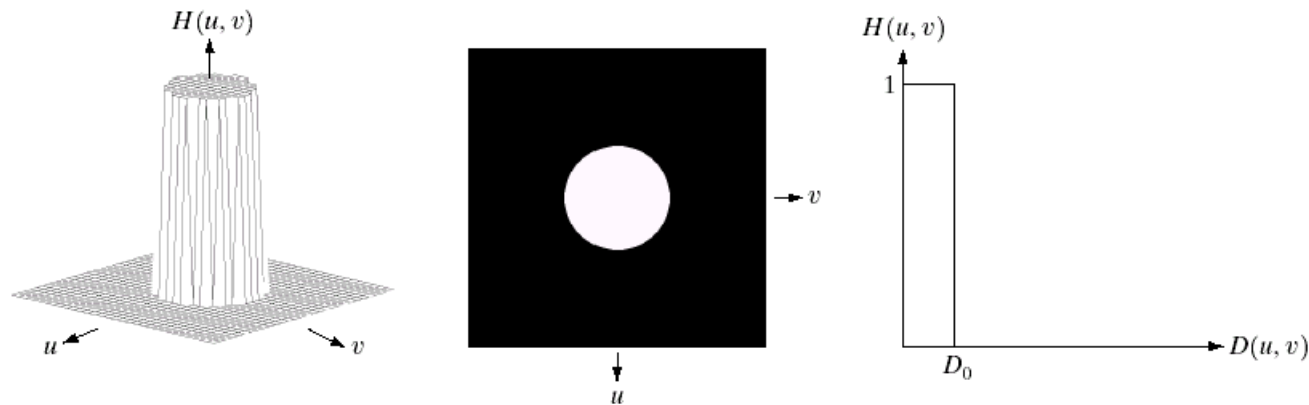
$D(u,v)$ is the distance from point (u,v) to the origin (center) of the frequency domain filter

$$H(u,v) = \begin{cases} 1 & \text{if } D(u,v) \leq D_0 \\ 0 & \text{if } D(u,v) > D_0 \end{cases}$$

Usually, the image to be filtered is even-sized, in this case, the center of the filter is $(M/2, N/2)$. Then the distance $D(u,v)$ can be obtained by

$$D(u,v) = [(u - M/2)^2 + (v - N/2)^2]^{1/2}$$

IDEAL LOW-PASS (LP) FILTERING



a b c

FIGURE 4.10 (a) Perspective plot of an ideal lowpass filter transfer function. (b) Filter displayed as an image. (c) Filter radial cross section.

How to determine the cutoff frequency D_0 ?

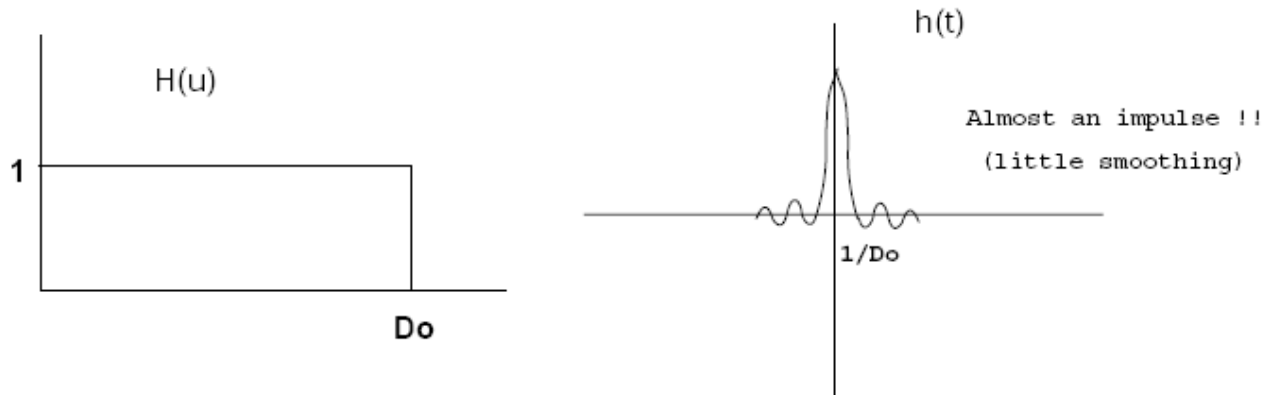
One way to do this is to compute circles that enclose specified amounts of total image power P_T .

$$P_T = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} |F(u, v)|^2$$

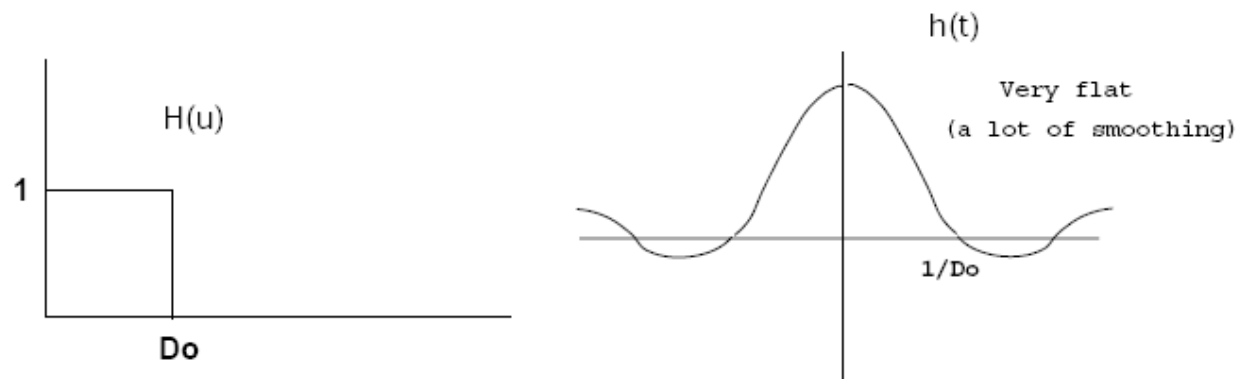
$$\left(\sum_{D(u,v) < D_0} |F(u, v)|^2 \right) / P_T \leq \alpha$$

HOW DOES D_0 CONTROL SMOOTHING? (CONT'D)

D_0 controls the amount of blurring



$r=78$ (99%)

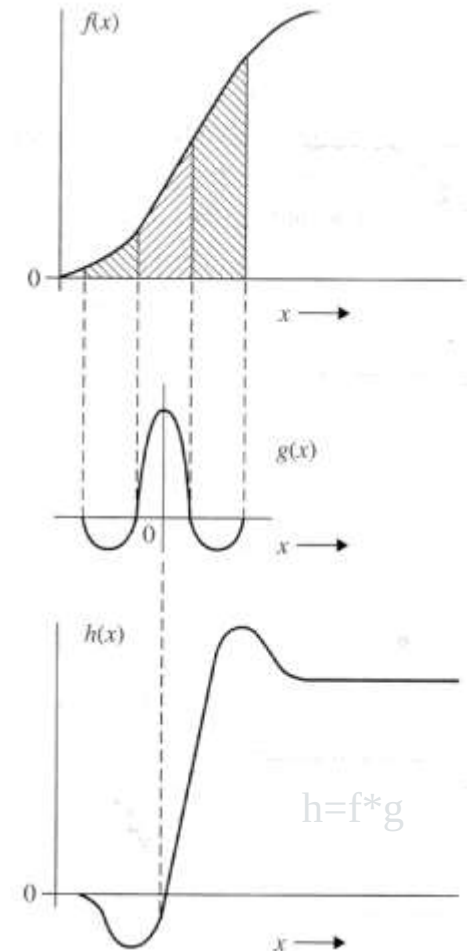


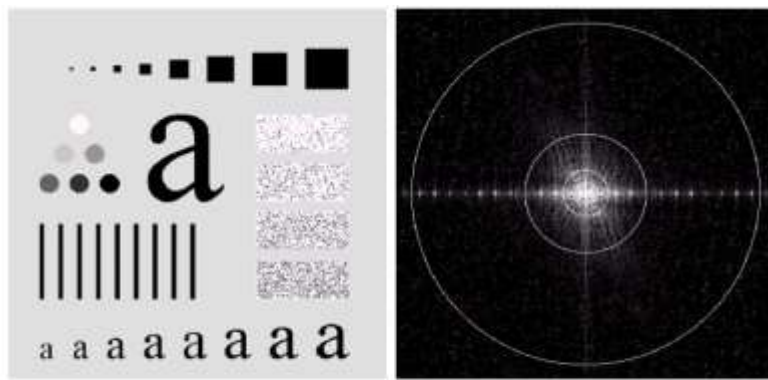
$r=8$ (90%)



RINGING EFFECT

Sharp cutoff frequencies produce an overshoot of image features whose frequency is close to the cutoff frequencies (**ringing effect**).





a b

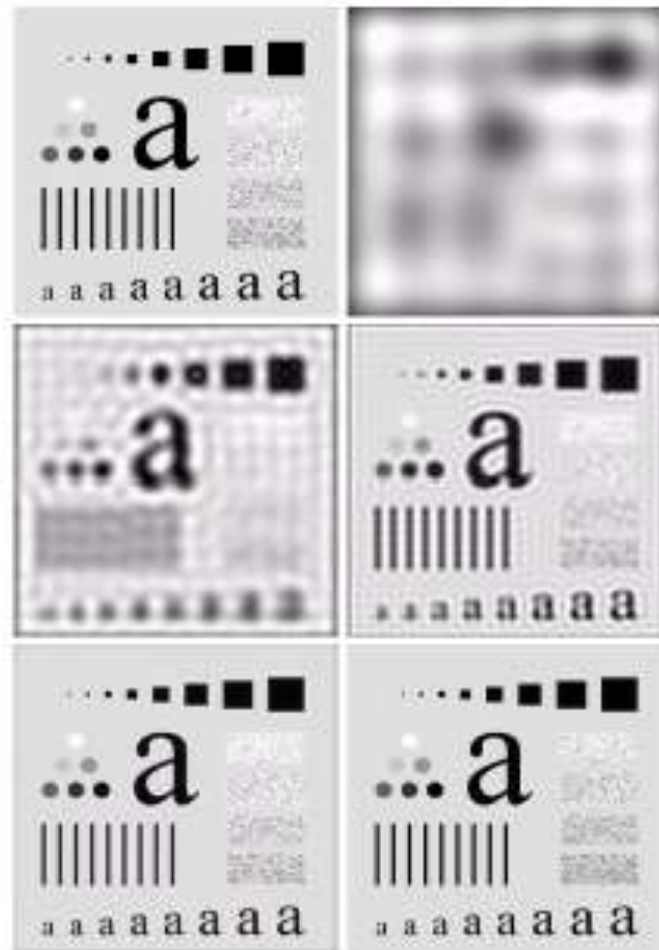
FIGURE 4.11 (a) An image of size 500×500 pixels and (b) its Fourier spectrum. The superimposed circles have radii values of 5, 15, 30, 80, and 230, which enclose 92.0, 94.6, 96.4, 98.0, and 99.5% of the image power, respectively.

Ringing effect is clear in most cases except for the last one.
Ringing effect is the consequence of applying ideal lowpass filters

Ringing effect is clear in most cases except for the last one.
Ringing effect is the consequence of applying ideal lowpass filters

original

15 pixels
94.6%
Ringing effect
due to ideal filter
behavior



5 pixels
92.0%
Useless,
severe blurring
(8% → detail)

30 pixels
96.4%
Ringing effect
due to ideal filter
behavior

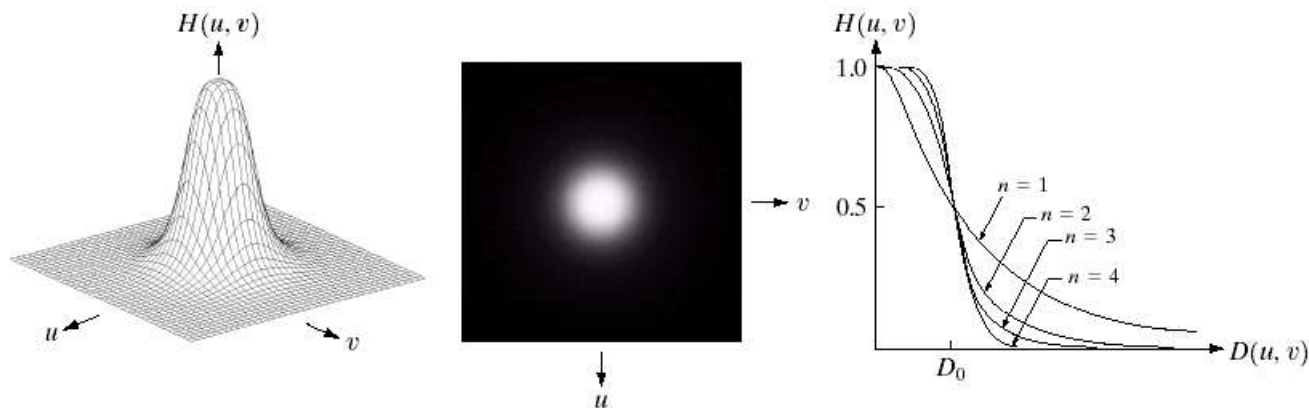
80 pixels
98.9%

230 pixels
99.5%

BUTTERWORTH LOWPASS FILTERS

A butterworth lowpass filter (BLPF) of order n with cutoff frequency at a distance D_0 from the origin is given by the following transfer function

- $$H(u, v) = \frac{1}{1 + [D(u, v) / D_0]^{2n}}$$
- BLPF does not have a sharp discontinuity
- For BLPF, the cutoff frequency is defined as the frequency at which the transfer function has value which is half of the maximum



a b c

FIGURE 4.14 (a) Perspective plot of a Butterworth lowpass filter transfer function. (b) Filter displayed as an image. (c) Filter radial cross sections of orders 1 through 4.

EXAMPLES OF APPLICATION OF BLPF

- Same order but with different cutoff frequencies
 - The larger the cutoff frequency, the more details are reserved

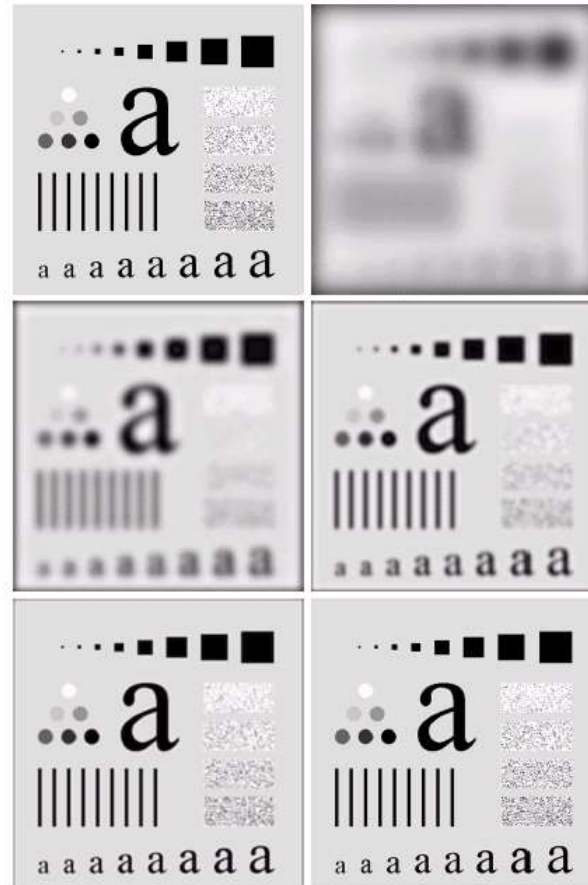


FIGURE 4.15 (a) Original image. (b)–(f) Results of filtering with BLPFs of order 2, with cutoff frequencies at radii of 5, 15, 30, 80, and 230, as shown in Fig. 4.11(b). Compare with Fig. 4.12.

BUTTERWORTH LPF

Provide a smooth transition between low and high frequencies

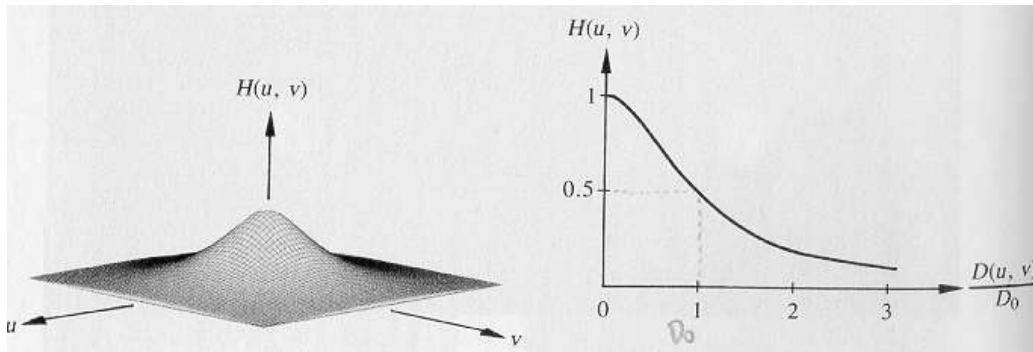
Reduce the ringing effect caused by the ILPF

Transfer function of Butterworth LPF (order n , cutoff frequency D_0):

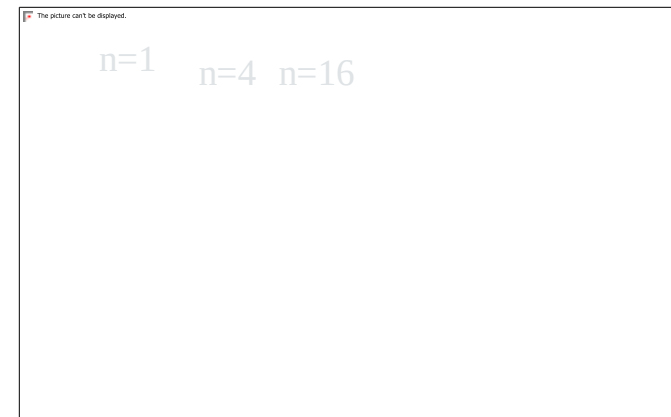
$$H(u, v) = \frac{1}{1 + \left[\frac{D(u, v)}{D_0} \right]^{2n}} = \frac{1}{1 + \left[\frac{(u - M/2)^2 + (v - N/2)^2}{D_0^2} \right]^n}$$

BUTTERWORTH LP FILTER (BLPF)

In practice, we use filters that attenuate high frequencies smoothly (e.g., **Butterworth** LP filter) → less ringing effect



$$H(u, v) = \frac{1}{1 + [\sqrt{u^2 + v^2}/D_0]^{2n}}$$



SPATIAL REPRESENTATION OF BLPFS

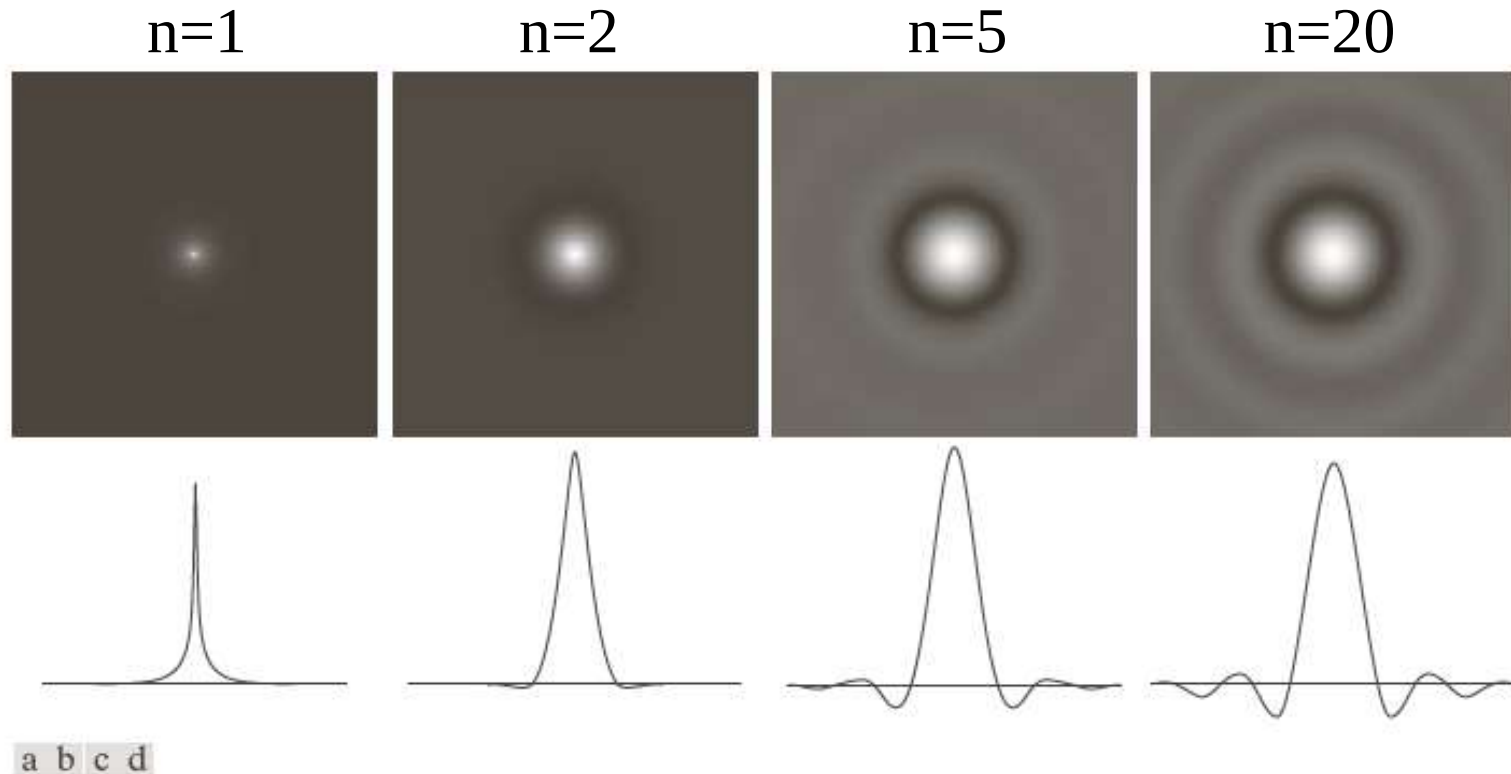


FIGURE 4.46 (a)–(d) Spatial representation of BLPFs of order 1, 2, 5, and 20, and corresponding intensity profiles through the center of the filters (the size in all cases is 1000×1000 and the cutoff frequency is 5). Observe how ringing increases as a function of filter order.

COMPARISON: IDEAL LP AND BLPF

ILPF

BLPF

$D_0=10, 30,$
60, 160,
460

$D_0=10, 30,$
60, 160,
460

$n=2$

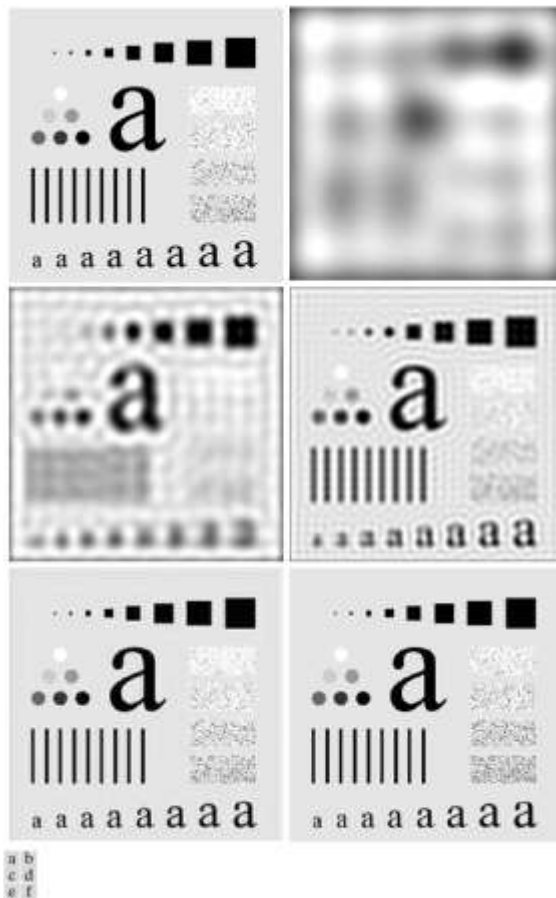


FIGURE 4.42 (a) Original image. (b)-(f) Results of filtering using ILPFs with cutoff frequencies set at radii values 10, 30, 60, 160, and 460, as shown in Fig. 4.41(b). The power removed by these filters was 13, 6.9, 4.3, 2.2, and 0.8% of the total, respectively.

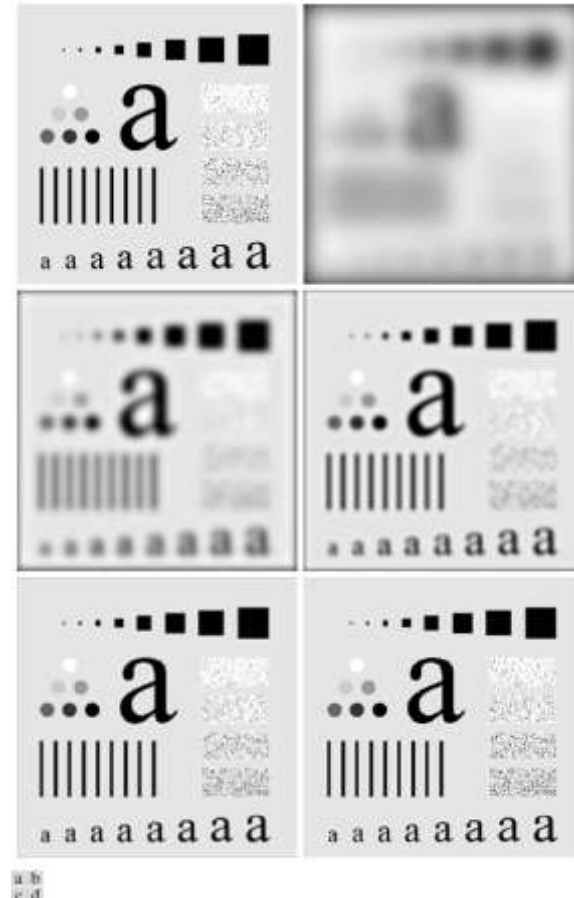


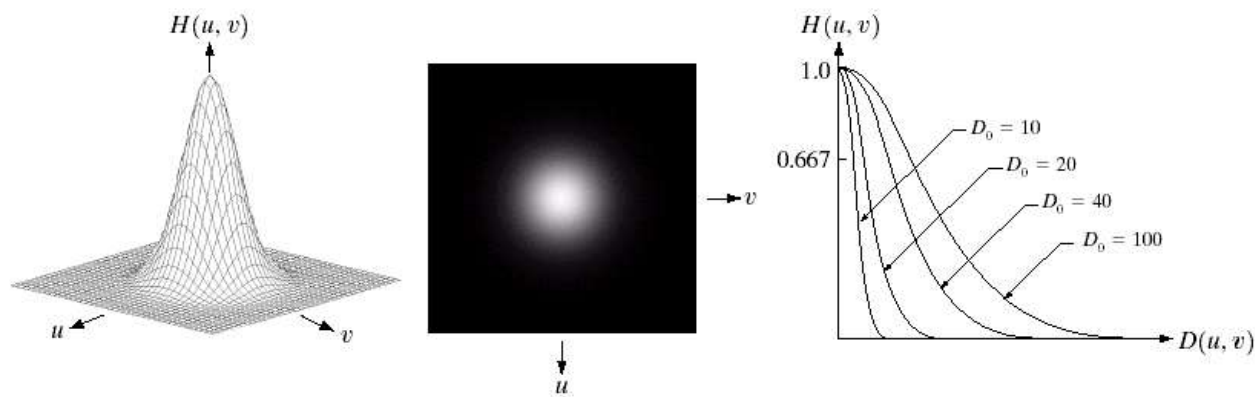
FIGURE 4.45 (a) Original image. (b)-(f) Results of filtering using BLPFs of order 2, with cutoff frequencies at the radii shown in Fig. 4.41. Compare with Fig. 4.42.

GAUSSIAN LOWPASS FILTERS

GLPF is given by the following (centered) transfer function

$$H(u, v) = e^{-[(u-u_0)^2 + (v-v_0)^2]/2\sigma^2} = e^{-D^2(u, v)/2D_0^2}$$

- (u_0, v_0) is the center of the transfer function
- It is $[M/2, N/2]$ if M, N are even and $[(M+1)/2, (N+1)/2]$ if M, N are odd numbers
- Dose GLPF suffer from the ringing effect?



a b c

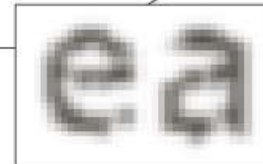
FIGURE 4.17 (a) Perspective plot of a GLPF transfer function. (b) Filter displayed as an image. (c) Filter radial cross sections for various values of D_0 .

Example: smoothing by GLPF (1)

Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.



Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.



a b

FIGURE 4.49

(a) Sample text of low resolution (note broken characters in magnified view).
(b) Result of filtering with a GLPF (broken character segments were joined).

Examples of smoothing by GLPF (2)

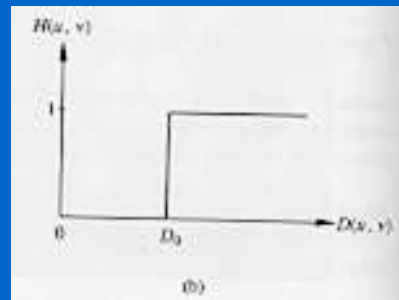


FIGURE 4.50 (a) Original image (784×732 pixels). (b) Result of filtering using a GLPF with $D_0 = 100$. (c) Result of filtering using a GLPF with $D_0 = 80$. Note the reduction in fine skin lines in the magnified sections in (b) and (c).

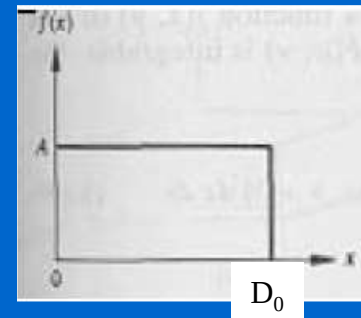
High-Pass filtering

- A high-pass filter can be obtained from a low-pass filter using:

$$H_{HP}(u, v) = 1 - H_{LP}(u, v)$$

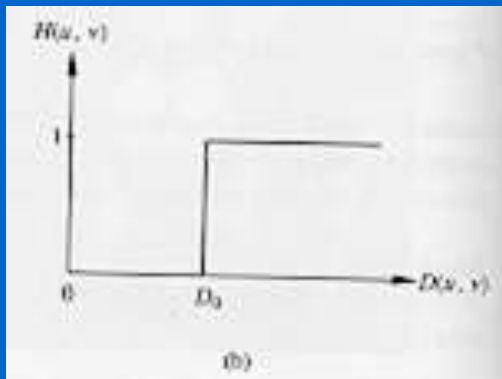


= 1 -

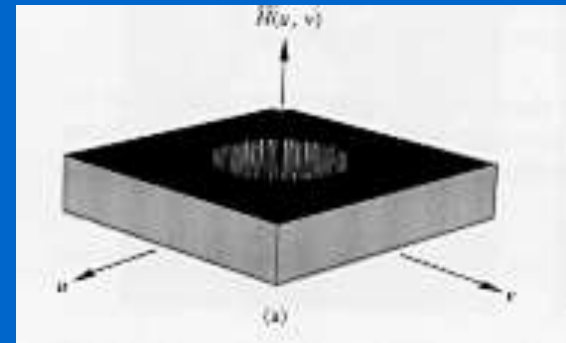


High-pass filtering (cont'd)

- Preserves high frequencies, attenuates low frequencies.



$$H(u, v) = \begin{cases} 1 & \text{if } u \geq D_0 \\ 0 & \text{otherwise} \end{cases}$$

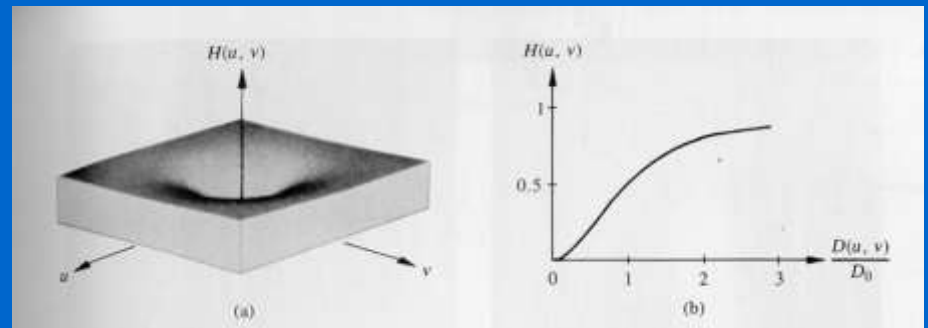


$$H(u, v) = \begin{cases} 1 & \text{if } u^2 + v^2 \geq D_0^2 \\ 0 & \text{otherwise} \end{cases}$$

Butterworth high pass filter (BHPF)

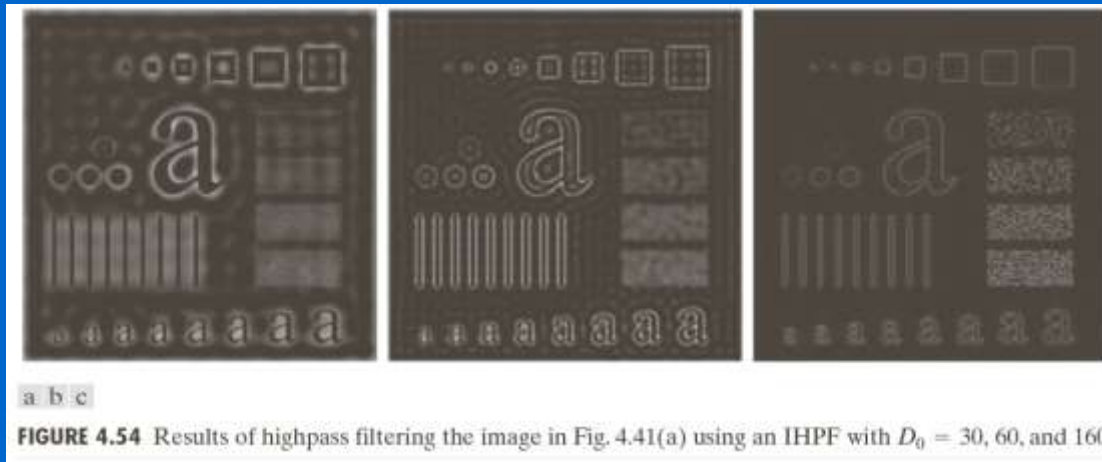
- In practice, we use filters that attenuate low frequencies smoothly (e.g., **Butterworth** HP filter) → less ringing effect

$$H(u, v) = \frac{1}{1 + [D_0/\sqrt{u^2 + v^2}]^{2n}}$$



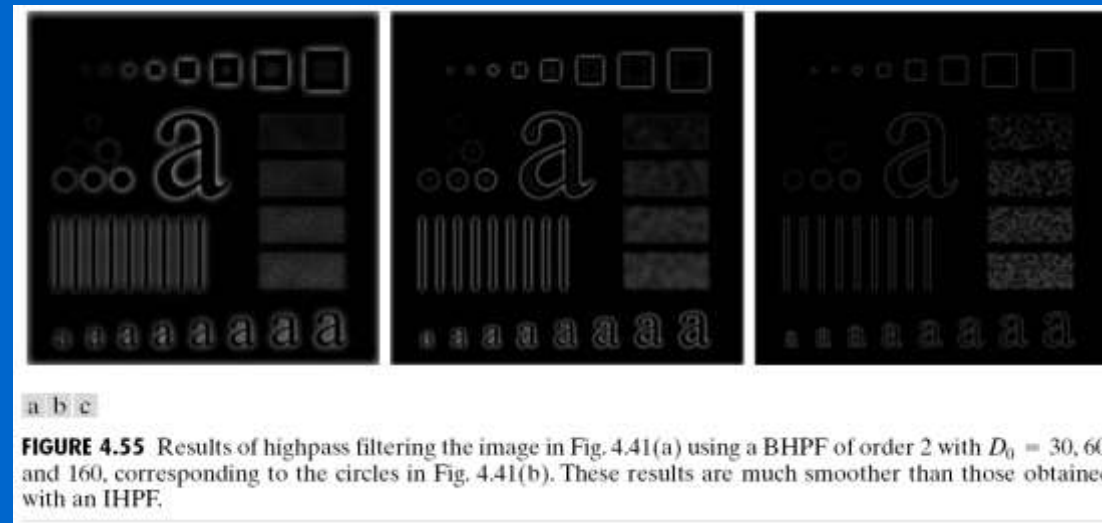
Comparison: IHPF and BHPF

IHPF



$D_0=30,60,160$

BHPF



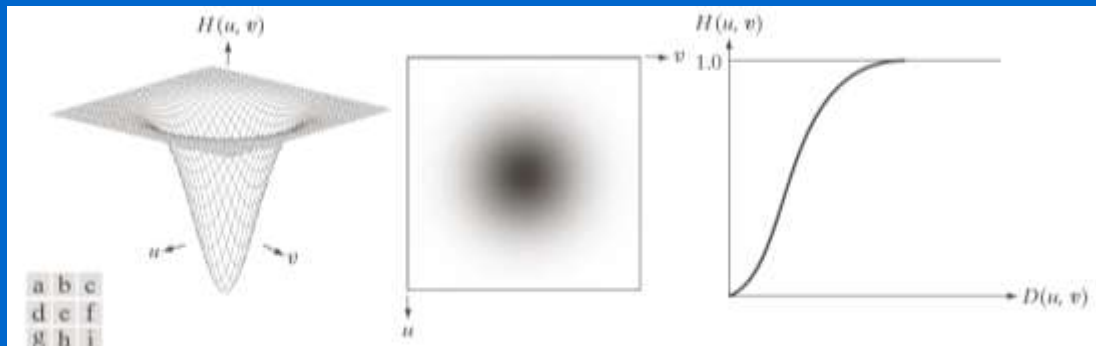
$D_0=30,60,160$

$n=2$

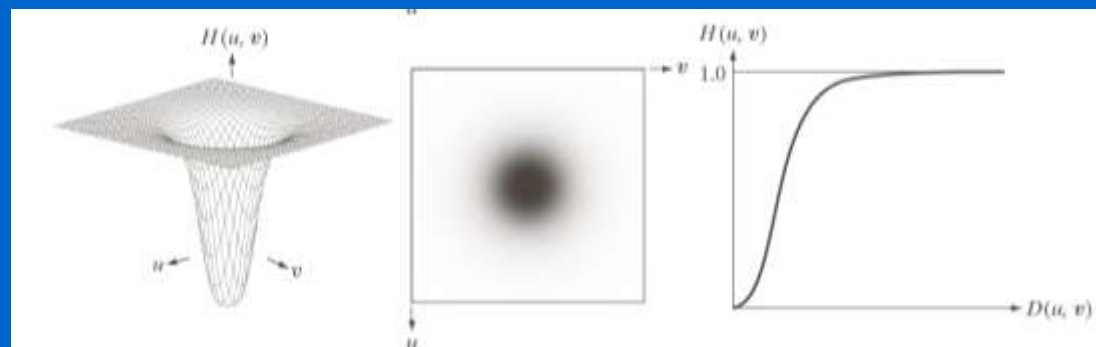
Gaussian HP filter

A 2-D Gaussian highpass filter (GHPL) is defined as

$$H(u, v) = 1 - e^{-(u^2 + v^2)/2D_0^2}$$



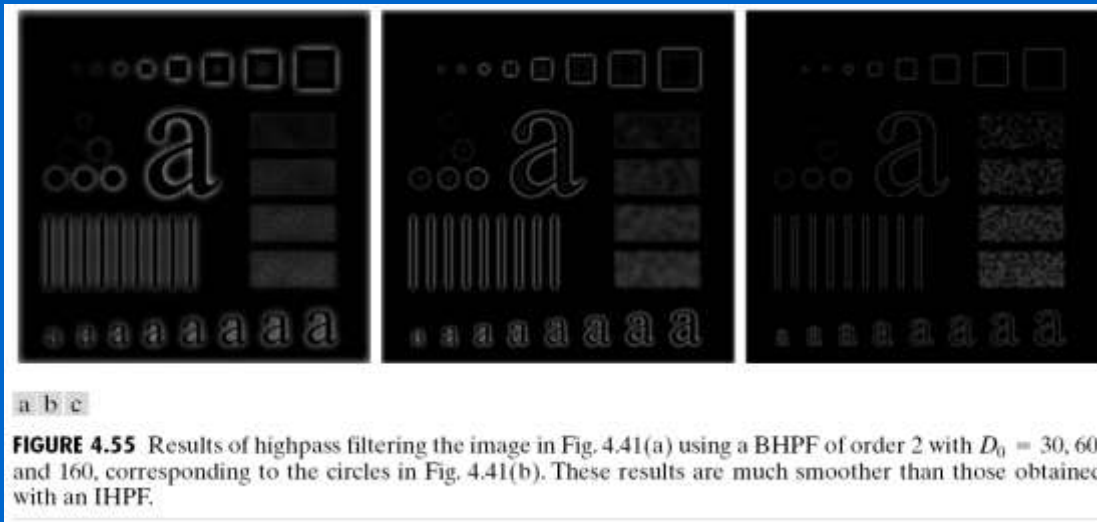
GHPF



BHPF

Comparison: BHPF and GHPF

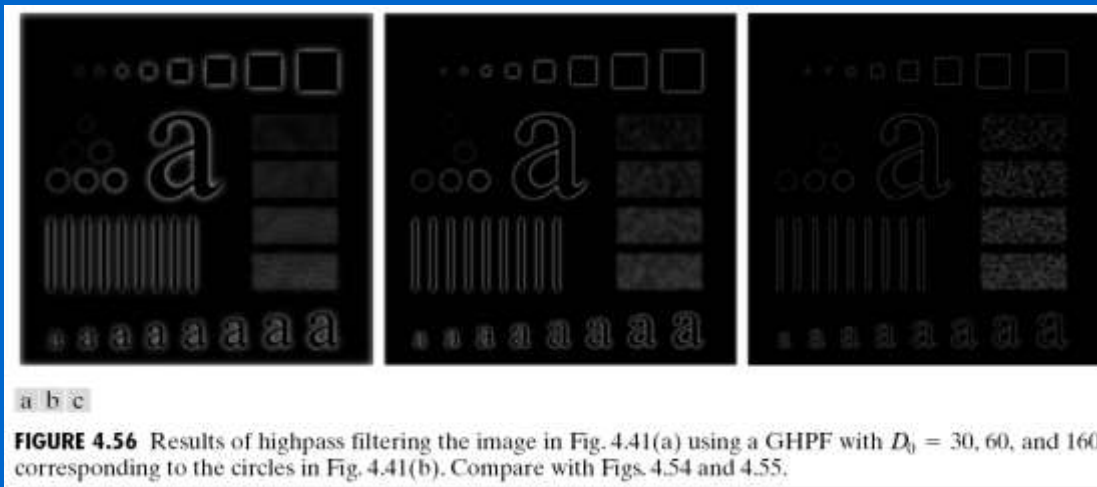
BHPF



$D_0=30,60,160$

$n=2$

GHPF



$D_0=30,60,160$

HOMOMORPHIC FILTERING

Many times, we want to remove shading effects from an image (i.e., due to uneven illumination)

- Enhance high frequencies
- Attenuate low frequencies but preserve fine detail.



HOMOMORPHIC FILTERING (CONT'D)

Consider the following model of image formation:

$$f(x, y) = i(x, y) r(x, y)$$

In general, the illumination component $i(x, y)$ varies **slowly** and affects **low** frequencies mostly.

In general, the reflection component $r(x, y)$ varies **faster** and affects **high** frequencies mostly.

HOW ARE FREQUENCIES MIXED TOGETHER?

$$f(x, y) = i(x, y) r(x, y) \quad \Rightarrow \quad F(u, v) = I(u, v) * R(u, v)$$

$$F(u, v)H(u, v) = [I(u, v) * R(u, v)]H(u, v)$$

CAN WE SEPARATE THEM?

Idea:

$$f(x, y) = i(x, y) r(x, y)$$

$$\ln(f(x, y)) = \ln(i(x, y)) + \ln(r(x, y))$$

STEPS OF HOMOMORPHIC FILTERING

(1) Take $\ln(f(x, y)) = \ln(i(x, y)) + \ln(r(x, y))$

(2) Apply FT: $F(\ln(f(x, y))) = F(\ln(i(x, y))) + F(\ln(r(x, y)))$

or $Z(u, v) = \text{Illum}(u, v) + \text{Refl}(u, v)$

(3) Apply $H(u, v)$

$$Z(u, v)H(u, v) = \text{Illum}(u, v)H(u, v) + \text{Refl}(u, v)H(u, v)$$

STEPS OF HOMOMORPHIC FILTERING (CONT'D)

(4) Take Inverse FT.

$$F^{-1}(Z(u, v)H(u, v)) = F^{-1}(\text{Illum}(u, v)H(u, v)) + F^{-1}(\text{Refl}(u, v)H(u, v))$$

$$s(x, y) = i'(x, y) + r'(x, y)$$

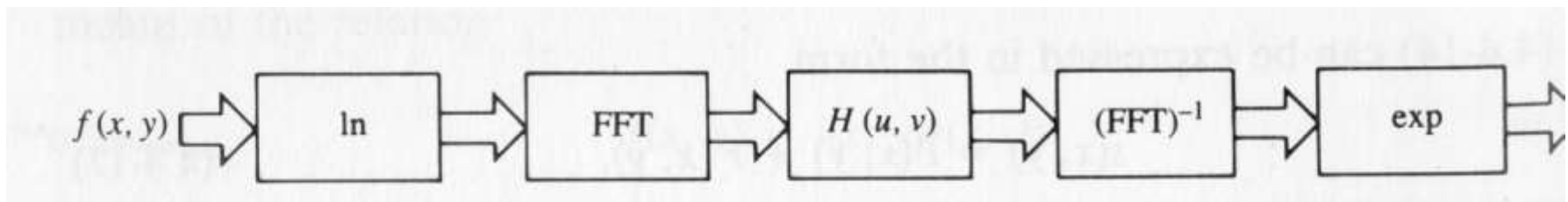
or

$$e^{s(x, y)} = e^{i'(x, y)} e^{r'(x, y)}$$

$$g(x, y) = i_0(x, y) r_0(x, y)$$

(5) Take exp()

or



EXAMPLE USING HIGH-FREQUENCY EMPHASIS

$$H(u, v) = (\gamma_H - \gamma_L) \left[1 - e^{-c[(u^2 + v^2)/D_0^2]} \right] + \gamma_L$$

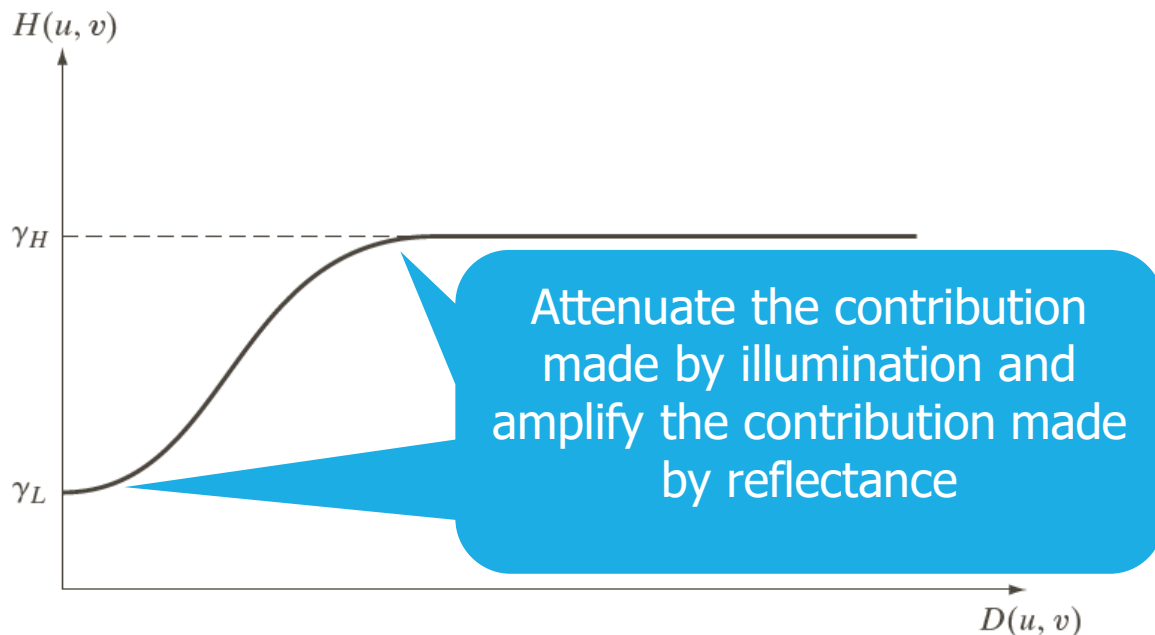


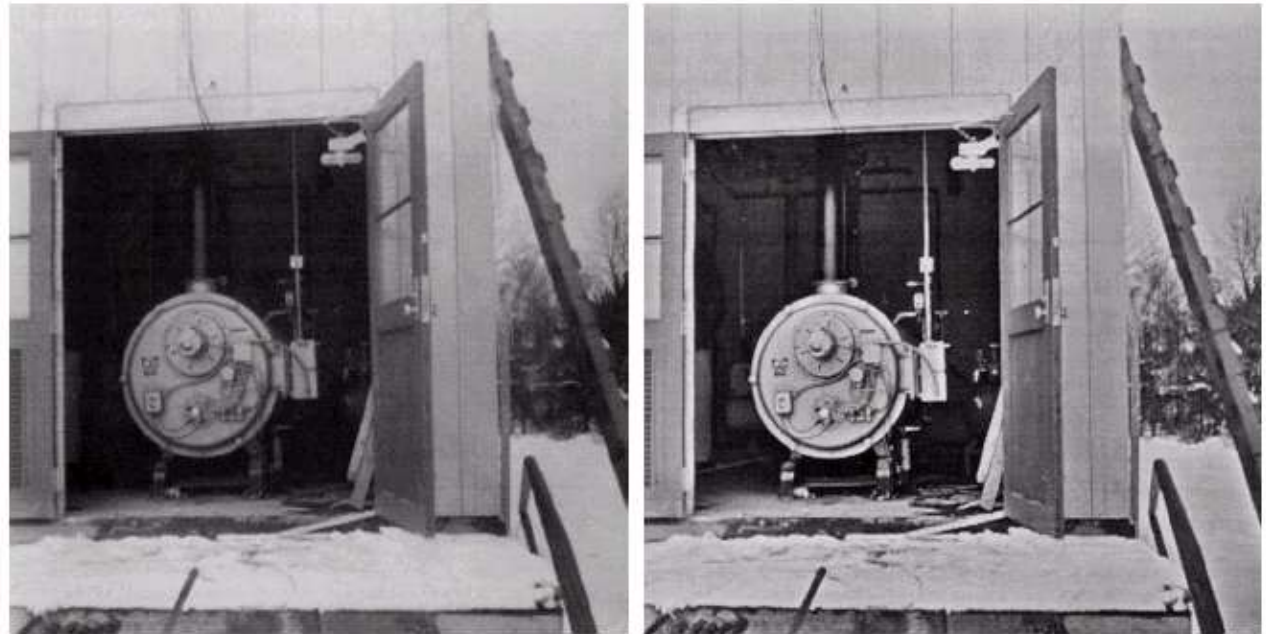
FIGURE 4.61 Radial cross section of a circularly symmetric homomorphic filter function. The vertical axis is at the center of the frequency rectangle and $D(u, v)$ is the distance from the center.

HOMOMORPHIC FILTERING: EXAMPLE

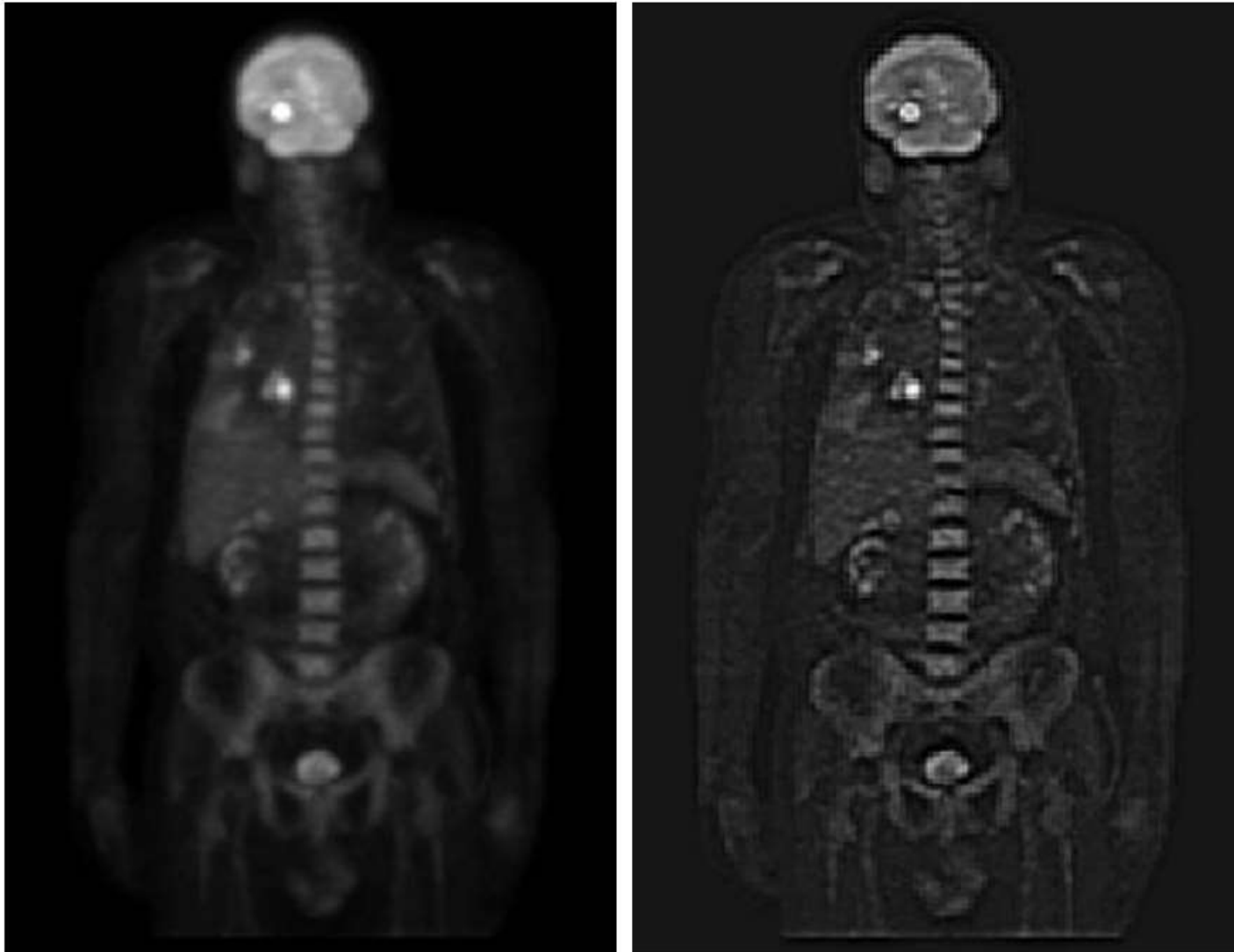
a b

FIGURE 4.33

(a) Original image. (b) Image processed by homomorphic filtering (note details inside shelter). (Stockham.)



HOMOMORPHIC FILTERING: EXAMPLE

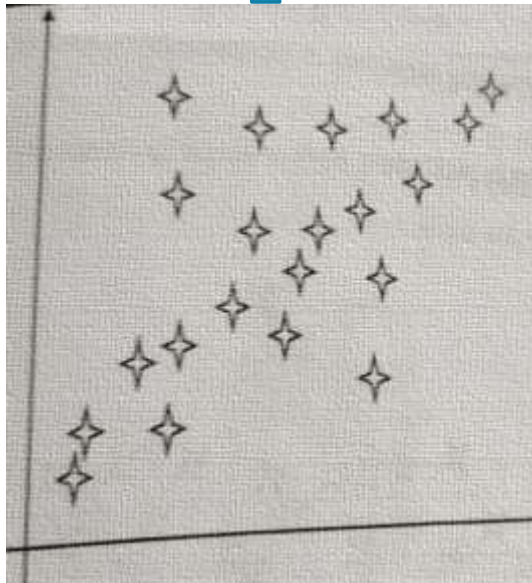


a b

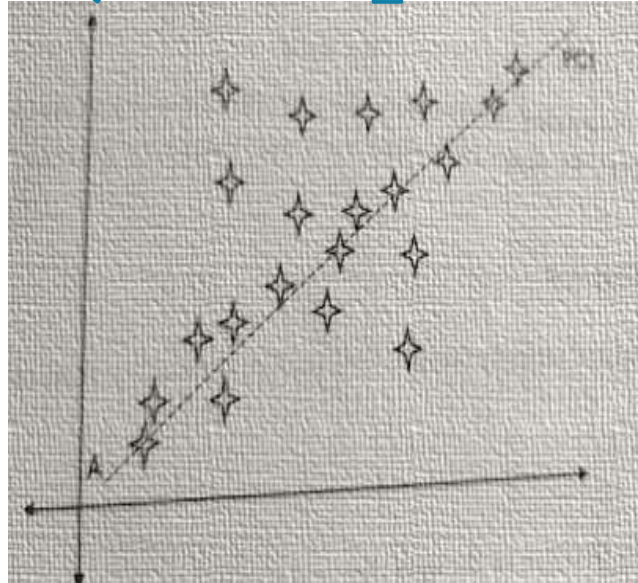
FIGURE 4.62

(a) Full body PET scan. (b) Image enhanced using homomorphic filtering. (Original image courtesy of Dr. Michael E. Casey, CTI PET Systems.)

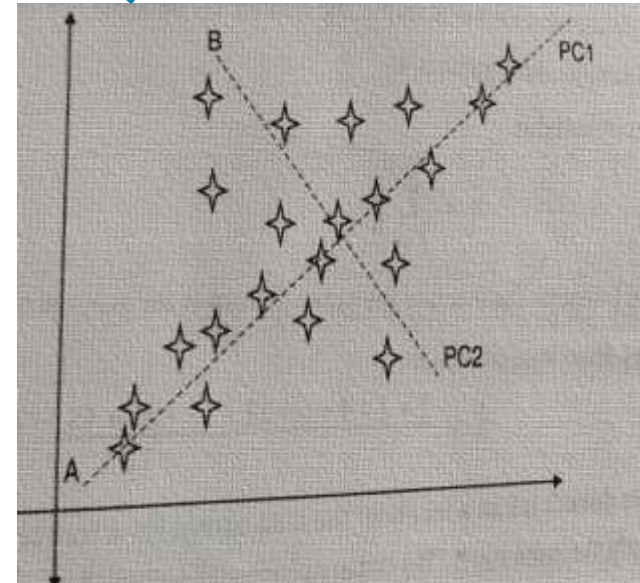
PCA STEPS:



Centre the data by subtracting off the mean



Choose the direction with largest variation, place an axis in that direction



Look at the remaining variations, find another axis that is orthogonal to the first axis and covers maximum variations

STANDARD DEVIATION

Standard deviation, is simply the square root of the average square distance of data points to the mean. In the equation below, the numerator contains the sum of the differences between each datapoint and the mean, and the denominator is simply the number of data points (minus one), producing the average distance.

$$s = \sqrt{\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{(n - 1)}}$$

VARIANCE

Variance is simply
as s^2

$$var(X) = \frac{\sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})}{(n - 1)}$$

is often expressed

For both variance and standard deviation, squaring the differences between data points and the mean makes them positive, so that values above and below the mean don't cancel each other out.

COVARIANCE

Variance and Covariance are a measure of the “spread” of a set of points around their center of mass (mean)

- Variance – measure of the deviation from the mean for points in one dimension e.g. heights
- Covariance as a measure of how much each of the dimensions vary from the mean with respect to each other.
- Covariance is measured between 2 dimensions to see if there is a relationship between the 2 dimensions e.g. number of hours studied & marks obtained.
- The covariance between one dimension and itself is the variance

COVARIANCE

So, if you had a 3-dimensional data set (x,y,z) , then you could measure the covariance between the x and y dimensions, the y and z dimensions, and the x and z dimensions.

Measuring the covariance between x and x , or y and y , or z and z would give you the variance of the x , y and z dimensions respectively.

$$\text{covariance}(X,Y) = \frac{\sum_{i=1}^n (\bar{X}_i - \bar{X})(\bar{Y}_i - \bar{Y})}{(n-1)}$$

COVARIANCE MATRIX

Representing Covariance between dimensions as a matrix e.g. for 3 dimensions:

$$C = \begin{bmatrix} \text{cov}(x, x) & \text{cov}(x, y) & \text{cov}(x, z) \\ \text{cov}(y, x) & \text{cov}(y, y) & \text{cov}(y, z) \\ \text{cov}(z, x) & \text{cov}(z, y) & \text{cov}(z, z) \end{bmatrix}$$

- Diagonal is the variances of x, y and z
- $\text{cov}(x, y) = \text{cov}(y, x)$ hence matrix is symmetrical about the diagonal
- N-dimensional data will result in NxN covariance matrix

COVARIANCE

Exact value is not as important as it's sign.

(+) A positive value of covariance indicates both dimensions increase or decrease together e.g. as the number of hours studied increases, the marks in that subject increase.

(-) A negative value indicates while one increases the other decreases, or vice-versa e.g. time spent on social media vs performance in exam.

(zero) If covariance is zero: the two dimensions are independent of each other e.g. heights of students vs the marks obtained in a subject

COVARIANCE

Why bother with calculating covariance when we could just plot the 2 values to see their relationship?

Ans:

Covariance calculations are used to find relationships between dimensions in high dimensional data sets (usually greater than 3) where visualization is difficult.

EIGEN VECTORS

$M = \begin{bmatrix} 2 & 3 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 11 \\ 5 \end{bmatrix}$ The resulting vector is not integer multiple of the original vector

$N = \begin{bmatrix} 2 & 3 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 12 \\ 8 \end{bmatrix} = 4 \times \begin{bmatrix} 3 \\ 2 \end{bmatrix}$ The resulting vector is exactly 4 times the original vector. This is called eigenvector.

We can transform and change matrices into new vectors by multiplying a matrix with a vector. The multiplication of the matrix by a vector computes a new vector. This is the transformed vector.

EIGENVECTOR

If the new transformed vector is just a scaled form of the original vector then the original vector is known to be an **eigenvector** of the original matrix

An eigenvector is a vector that does not change when a transformation is applied to it, except that it becomes a scaled version of the original vector.

Eigenvalue—The scalar that is used to transform (stretch) an Eigenvector.

EIGENVECTOR AND EIGENVALUE

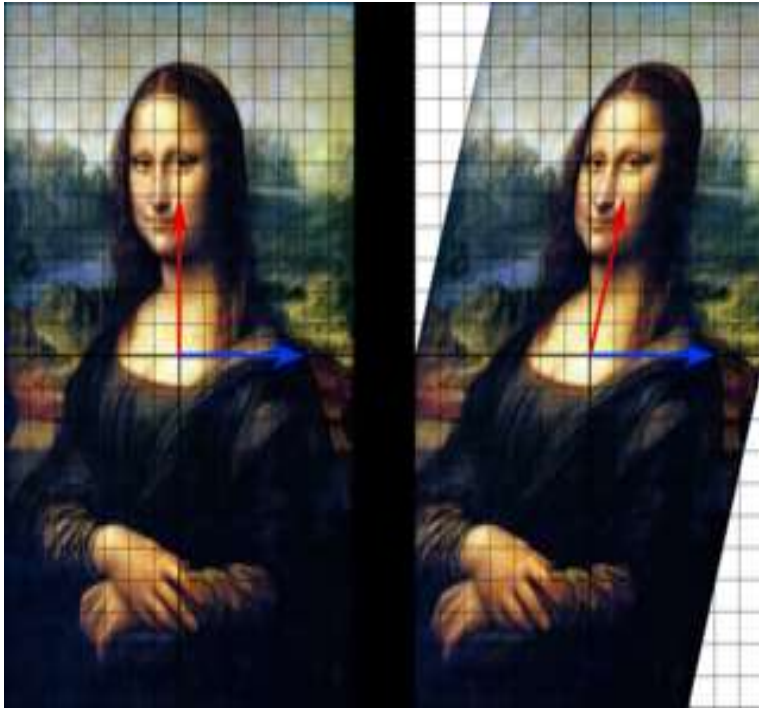
An eigenvector **does not change direction** in a transformation:

In this shear mapping the red arrow changes direction, but the blue arrow does not. The blue arrow is an eigenvector of this shear mapping because it does not change direction, and since its length is unchanged, its eigenvalue is 1.

1 means no change,

2 means doubling in length,

-1 means pointing backwards along the eigenvector's direction



THE MATHEMATICS

For a square matrix **A**, an Eigenvector and Eigenvalue make this equation true:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

Matrix Eigenvector Eigenvalue

A diagram showing the equation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Below the equation, the word 'Matrix' has an arrow pointing to $\mathbf{A}$. The word 'Eigenvector' has an arrow pointing to $\mathbf{v}$ on the left and another arrow pointing to $\mathbf{v}$ on the right. The word 'Eigenvalue' has an arrow pointing to λ .

$\mathbf{A}\mathbf{v}$ gives us:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -6 \times 1 + 3 \times 4 \\ 4 \times 1 + 5 \times 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

$\lambda\mathbf{v}$ gives us :

$$6 \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

Yes they are equal! So $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ as promised.

HOW DO WE FIND THESE EIGEN THINGS?

We start by finding the **eigenvalue**:

we know this equation must be true: $Av = \lambda v$

Now let us put in an identity matrix so we are dealing with matrix-vs-matrix:

$$Av = \lambda Iv$$

Bring all to left hand side:

$$Av - \lambda Iv = 0$$

If v is non-zero then we can solve for λ using just the determinant:

$$| A - \lambda I | = 0$$

Let's try that equation on our previous example:

Example: Solve for λ :

Start with $|A - \lambda I| = 0$

$$\left| \begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right| = 0$$

Which is:

$$\left| \begin{array}{cc} -6-\lambda & 3 \\ 4 & 5-\lambda \end{array} \right| = 0$$

Calculating that determinant gets:

$$(-6-\lambda)(5-\lambda) - 3 \times 4 = 0$$

Which then gets us this

$$\lambda^2 + \lambda - 42 = 0$$

$$\lambda = -7 \text{ or } 6$$

Example (continued): Find the Eigenvector for the Eigenvalue $\lambda = 6$:

Start with:

$$Av = \lambda v$$

Put in the values we know:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 6 \begin{bmatrix} x \\ y \end{bmatrix}$$

After multiplying we get these two equations:

$$-6x + 3y = 6x$$

$$4x + 5y = 6y$$

Bringing all to left hand side:

$$-12x + 3y = 0$$

$$4x - 1y = 0$$

Either equation reveals that $y = 4x$, so the **eigenvector** is any non-zero multiple of this:

$$\begin{bmatrix} 1 \\ 4 \end{bmatrix}$$

And we get the solution shown at the top of the page:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -6 \times 1 + 3 \times 4 \\ 4 \times 1 + 5 \times 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

... and also ...

$$6 \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

So $Av = \lambda v$

PCA

Principal components analysis (PCA) is a technique that can be used to simplify a dataset

- It is a linear transformation that chooses a new coordinate system for the data set such that greatest variance by any projection of the data set comes to lie on the first axis (then called the first principal component), the second greatest variance on the second axis, and so on.
- PCA can be used for reducing dimensionality by eliminating the later principal components.

STEPS INVOLVED IN THE PCA

Step 1: Standardize the dataset.

Step 2: Calculate the covariance matrix for the features in the dataset.

Step 3: Calculate the eigenvalues and eigenvectors for the covariance matrix.

Step 4: Sort eigenvalues and their corresponding eigenvectors.

Step 5: Pick k eigenvalues and form a matrix of eigenvectors.

Step 6: Transform the original matrix.

EXAMPLE

1. Standardize the Dataset

Assume we have the below dataset which has 4 features and a total of 5 training examples.

f1	f2	f3	f4
1	2	3	4
5	5	6	7
1	4	2	3
5	3	2	1
8	1	2	2

EXAMPLE

First, we need to standardize the dataset and for that, we need to calculate each feature.

$$x_{new} = \frac{x - \mu}{\sigma}$$

After applying the formula for each feature in the dataset is transformed as below:

f1	f2	f3	f4
1	2	3	4
5	5	6	7
1	4	2	3
5	3	2	1
8	1	2	2

	f1	f2	f3	f4
μ =	4	3	3	3.4
σ =	3	1.58114	1.73205	2.30217

f1	f2	f3	f4
-1	-0.63246	0	0.26062
0.33333	1.26491	1.73205	1.56374
-1	0.63246	-0.57735	-0.17375
0.33333	0	-0.57735	-1.04249
1.33333	-1.26491	-0.57735	-0.60812

$$s = \sqrt{\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{(n-1)}}$$

EXAMPLE

2. Calculate the covariance matrix for the whole dataset

The formula to calculate the covariance matrix:

For Population

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x}) * (y_i - \bar{y})}{N}$$

For Sample

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x}) * (y_i - \bar{y})}{(N-1)}$$

	f1	f2	f3	f4
f1	var(f1)	cov(f1,f2)	cov(f1,f3)	cov(f1,f4)
f2	cov(f2,f1)	var(f2)	cov(f2,f3)	cov(f2,f4)
f3	cov(f3,f1)	cov(f3,f2)	var(f3)	cov(f3,f4)
f4	cov(f4,f1)	cov(f4,f2)	cov(f4,f3)	var(f4)

Since we have standardized the dataset, so the **mean for each feature is 0** and the **standard deviation is 1**.

EXAMPLE

f1	f2	f3	f4
-1	-0.63246	0	0.26062
0.33333	1.26491	1.73205	1.56374
-1	0.63246	-0.57735	-0.17375
0.33333	0	-0.57735	-1.04249
1.33333	-1.26491	-0.57735	-0.60812

	f1	f2	f3	f4
f1	var(f1)	cov(f1,f2)	cov(f1,f3)	cov(f1,f4)
f2	cov(f2,f1)	var(f2)	cov(f2,f3)	cov(f2,f4)
f3	cov(f3,f1)	cov(f3,f2)	var(f3)	cov(f3,f4)
f4	cov(f4,f1)	cov(f4,f2)	cov(f4,f3)	var(f4)

$$\text{var}(f1) = ((-1.0-0)^2 + (0.33-0)^2 + (-1.0-0)^2 + (0.33-0)^2 + (1.33-0)^2)/5$$

var (f1) = 0.8

$$\text{cov}(f1,f2) = ((-1.0-0)*(-0.632456-0) + (0.33-0)*(1.264911-0) + (-1.0-0)*(0.632456-0) + (0.33-0)*(0.000000-0) + (1.33-0)*(-1.264911-0))/5$$

cov(f1,f2) = -0.25298

EXAMPLE

In the similar way we can calculate the other covariances and which will result in the below covariance matrix

	f1	f2	f3	f4
f1	0.8	-0.25298	0.03849	-0.14479
f2	-0.25298	0.8	0.51121	0.4945
f3	0.03849	0.51121	0.8	0.75236
f4	-0.14479	0.4945	0.75236	0.8

EXAMPLE

3. Calculate eigenvalues and eigen vectors.

$$Av - \lambda Iv = 0 ;$$

$$(A - \lambda I)v = 0$$

Since we already know v is a non-zero vector, only way this equation can be equal to zero, if

$$\det(A - \lambda I) = 0$$

EXAMPLE

$$\begin{bmatrix} 0.8 & -0.25298 & 0.03849 & -0.14479 \\ -0.25298 & 0.8 & 0.51121 & 0.4945 \\ 0.03849 & 0.51121 & 0.8 & 0.75236 \\ -0.14479 & 0.4945 & 0.75236 & 0.8 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = 0$$

	f1	f2	f3	f4
f1	$0.8 - \lambda$	-0.25298	0.03849	-0.14479
f2	-0.25298	$0.8 - \lambda$	0.51121	0.4945
f3	0.03849	0.51121	$0.8 - \lambda$	0.75236
f4	-0.14479	0.4945	0.75236	$0.8 - \lambda$

Solving the above equation $|M| = 0$

$\lambda = 2.51579324, 1.0652885, 0.39388704, 0.02503121$

EXAMPLE

Eigenvectors:

$$\begin{pmatrix} 0.800000 - \lambda & -(0.252982) & 0.038490 & -(0.144791) \\ -(0.252982) & 0.800000 - \lambda & 0.511208 & 0.494498 \\ 0.038490 & 0.511208 & 0.800000 - \lambda & 0.752355 \\ -(0.144791) & 0.494498 & 0.752355 & 0.800000 - \lambda \end{pmatrix} \times \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix} = 0$$

For $\lambda = 2.51579324$, solving the above equation using Cramer's rule, the values for v vector are

$$v_1 = 0.16195986$$

$$v_2 = -0.52404813$$

$$v_3 = -0.58589647$$

$$v_4 = -0.59654663$$

EXAMPLE

Going by the same approach, we can calculate the eigen vectors for the other eigenvalues. **e1 e2 e3 e4 n vectors.**

0.161960	-0.917059	-0.307071	0.196162
-0.524048	0.206922	-0.817319	0.120610
-0.585896	-0.320539	0.188250	-0.720099
-0.596547	-0.115935	0.449733	0.654547

4. Sort eigenvalues and their corresponding eigenvectors.

Since eigenvalues are already sorted in this case so no need to sort them again.

EXAMPLE

5. Pick k eigenvalues and form a matrix of eigenvectors

If we choose the top 2 eigenvectors, the matrix will look like this:

e1	e2
0.161960	-0.917059
-0.524048	0.206922
-0.585896	-0.320539
-0.596547	-0.115935

6. Transform the original matrix.

Feature matrix * top k eigenvectors = Transformed Data

f1	f2	f3	f4		e1	e2		nf1	nf2
-1.000000	-0.632456	0.000000	0.260623		0.161960	-0.917059		0.014003	0.755975
0.333333	1.264911	1.732051	1.563740	*	-0.524048	0.206922	=	-2.556534	-0.780432
-1.000000	0.632456	-0.577350	-0.173749		-0.585896	-0.320539		-0.051480	1.253135
0.333333	0.000000	-0.577350	-1.042493		-0.596547	-0.115935		1.014150	0.000239
1.333333	-1.264911	-0.577350	-0.608121					1.579861	-1.228917
			(5,4)		(4,2)			(5,2)	

```

import numpy as np
import pandas as pd
A = np.matrix([[1,2,3,4],
               [5,5,6,7],
               [1,4,2,3],
               [5,3,2,1],
               [8,1,2,2]])

df = pd.DataFrame(A,columns = ['f1','f2','f3','f4'])
df_std = (df - df.mean()) / (df.std())
n_components = 2
from sklearn.decomposition import PCA
pca = PCA(n_components=n_components)
principalComponents = pca.fit_transform(df_std)
principalDf = pd.DataFrame(data=principalComponents,columns=['nf'+str(i+1) for i in range(n_components)])
print(principalDf)

```

```

      nf1      nf2
0 -0.014003  0.755975
1  2.556534 -0.780432
2  0.051480  1.253135
3 -1.014150  0.000239
4 -1.579861 -1.228917

```


WAVELET TRANSFORM

<https://medium.com/@koushikc2000/2d-discrete-wavelet-transformation-and-its-applications-in-digital-image-processing-using-matlab-1f5c68672de3>